

Regular Grammar

Noam Chomsky gave a Mathematical model of Grammar which is effective for writing computer languages

The four types of Grammar according to Noam Chomsky are:

Grammar Type	Grammar Accepted	Language Accepted	Automaton
TYPE-0	Unrestricted Grammar	Recursively Enumerable Language	Turing Machine
TYPE-1	Context Sensitive Grammar	Context Sensitive Language	Linear Bounded Automaton
TYPE-2	Context Free Grammar	Context Free Language	Pushdown Automata
TYPE-3	Regular Grammar	Regular Language	Finite State Automaton



Grammar:

A Grammar 'G' can be formally described using 4 tuples as $G = (V, T, S, P)$ where,

V = Set of Variables or Non-Terminal Symbols

T = Set of Terminal Symbols

S = Start Symbol

P = Production rules for Terminals and Non-Terminals

A production rule has the form $\alpha \rightarrow \beta$ where α and β are strings on $V \cup T$ and at least one symbol of α belongs to V.

Example: $G = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$



Grammar:

A Grammar 'G' can be formally described using 4 tuples as $G = (V, T, S, P)$ where,

V = Set of Variables or Non-Terminal Symbols

T = Set of Terminal Symbols

S = Start Symbol

P = Production rules for Terminals and Non-Terminals

A production rule has the form $\alpha \rightarrow \beta$ where α and β are strings on $V \cup T$ and at least one symbol of α belongs to V.

Example: $G = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$

$$V = \{S, A, B\}$$

$$T = \{a, b\}$$

$$S = S$$

$$P = S \rightarrow AB, A \rightarrow a, B \rightarrow b$$

Eg.



Grammar:

A Grammar 'G' can be formally described using 4 tuples as $G = (V, T, S, P)$ where,

V = Set of Variables or Non-Terminal Symbols

T = Set of Terminal Symbols

S = Start Symbol

P = Production rules for Terminals and Non-Terminals

A production rule has the form $\alpha \rightarrow \beta$ where α and β are strings on $V \cup T$ and at least one symbol of α belongs to V.

Example: $G = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$

$$V = \{S, A, B\}$$

$$T = \{a, b\}$$

$$S = S$$

$$P = S \rightarrow AB, A \rightarrow a, B \rightarrow b$$

Eg.

$$\begin{aligned} S &\rightarrow AB \\ &\rightarrow aB \\ &\rightarrow \underline{ab} \end{aligned}$$



Regular Grammar:

Regular Grammar can be divided into two types:

Right Linear Grammar

A grammar is said to be Right Linear if all productions are of the form

$$A \rightarrow xB$$

$$A \rightarrow x$$

where $A, B \in V$ and $x \in T$

Left Linear Grammar

A grammar is said to be Left Linear if all productions are of the form

$$A \rightarrow Bx$$

$$A \rightarrow x$$

where $A, B \in V$ and $x \in T$

Ex: $S \rightarrow abS \mid b$ - Right linear

$S \rightarrow Sbb \mid b$ - Left linear



Derivations from a Grammar

The set of all strings that can be derived from a Grammar is said to be the LANGUAGE generated from that Grammar

Example 1: Consider the Grammar $G1 = (\{S, A\}, \{a, b\}, S, \{S \rightarrow aAb, aA \rightarrow aaAb, A \rightarrow \epsilon\})$

$S \rightarrow \underline{a}Ab$ [by $S \rightarrow aAb$]
 $\rightarrow aa\underline{A}bb$ [by $aA \rightarrow aaAb$]
 $\rightarrow a aa\underline{A}bbb$ [by $aA \rightarrow aaAb$]
 $\rightarrow a aabbb$ [by $A \rightarrow \epsilon$]



$\rightarrow a a \underline{A} b b b$ [by $aA \rightarrow a a A b$]
 $\rightarrow a a b b b$ [by $A \rightarrow \epsilon$]

Example 2: $G_2 = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$

$S \rightarrow AB$
 $\rightarrow ab$

$L(G_2) = \{ab\}$

Example 3: $G_3 = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow aA|a, B \rightarrow bB|b\})$

$S \rightarrow AB$	$S \rightarrow AB$	$S \rightarrow AB$	$S \rightarrow AB$
$\rightarrow ab$	$\rightarrow aA bB$	$\rightarrow aAb$	$\rightarrow a bB$
	$\rightarrow aabb$	$\rightarrow aab$	$\rightarrow abb$

$L(G_3) = \{ab, a^2b^2, a^2b, ab^2, \dots\}$
 $= \{a^m b^n \mid m \geq 0 \text{ and } n \geq 0\}$



Context Free Language

In formal language theory, a Context Free Language is a language generated by some Context Free Grammar.

The set of all CFL is identical to the set of languages accepted by Pushdown Automata.

Context Free Grammar is defined by 4 tuples as $G = \{ V, \Sigma, S, P \}$ where

V = Set of Variables or Non-Terminal Symbols

Σ = Set of Terminal Symbols

S = Start Symbol

P = Production Rule

Context Free Grammar has Production Rule of the form

$$A \rightarrow \alpha$$

where, $\alpha \in \{V \cup \Sigma\}^*$ and $A \in V$



Context Free Language

In formal language theory, a Context Free Language is a language generated by some Context Free Grammar.

The set of all CFL is identical to the set of languages accepted by Pushdown Automata.

Context Free Grammar is defined by 4 tuples as $G = \{ V, \Sigma, S, P \}$ where

V = Set of Variables or Non-Terminal Symbols

Σ = Set of Terminal Symbols

S = Start Symbol

P = Production Rule

Context Free Grammar has Production Rule of the form
$A \rightarrow a$
where, $a \in \{V \cup \Sigma\}^*$ and $A \in V$

Example: For generating a language that generates equal number of a's and b's in the form $a^n b^n$, the Context Free Grammar will be defined as

$G = \{ (S, A), (a, b), (S \rightarrow aAb, A \rightarrow aAb | \epsilon) \}$



Context Free Grammar is defined by 4 tuples as $G = (V, \Sigma, S, P)$ where

V = Set of Variables or Non-Terminal Symbols

Σ = Set of Terminal Symbols

S = Start Symbol

P = Production Rule

Context Free Grammar has Production Rule of the form

$$A \rightarrow \alpha$$

where, $\alpha \in \{V \cup \Sigma\}^*$ and $A \in V$

Example: For generating a language that generates equal number of a's and b's in the form $a^n b^n$, the Context Free Grammar will be defined as

$$G = \{ (S, A), (a, b), (S \rightarrow aAb, A \rightarrow aAb | \epsilon) \}$$

$$S \rightarrow a \underline{A} b$$

$$\rightarrow a a \underline{A} b b \quad (\text{by } A \rightarrow aAb)$$

$$\rightarrow a a a \underline{A} b b b \quad (\quad)$$

$$\rightarrow a a a b b b \quad (\text{by } A \rightarrow \epsilon)$$

$$\rightarrow \underline{a}^3 \underline{b}^3 \Rightarrow a^n b^n$$



Method to find whether a String belongs to a Grammar or not

- 1) Start with the Start Symbol and choose the closest production that matches to the given string.
- 2) Replace the Variables with its most appropriate production. Repeat the process until the string is generated or until no other productions are left.

Example: Verify whether the Grammar $S \rightarrow 0B|1A$, $A \rightarrow 0|0S|1AA|^{\wedge}$, $B \rightarrow 1|1S|0BB$ generates the string 00110101



Example: Verify whether the Grammar $S \rightarrow aAb$, $A \rightarrow aAb \mid \epsilon$ generates the string $aabbb$

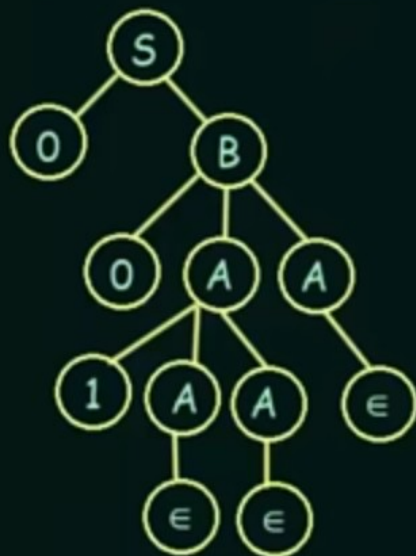
$S \rightarrow aAb$
 $\rightarrow aaAbb$ ($A \rightarrow aAb$)
 $\rightarrow aabb$ ($A \rightarrow \epsilon$)
 $\rightarrow aaaaAbbb$ ($A \rightarrow aAb$)
 $aaaaabb$ ($A \rightarrow \epsilon$) X



Derivation Tree

A Derivation Tree or Parse Tree is an ordered rooted tree that graphically represents the semantic information of strings derived from a Context Free Grammar

Example: For the Grammar $G = \{V, T, P, S\}$ where $S \rightarrow OB$, $A \rightarrow 1AA | \epsilon$, $B \rightarrow OAA$



Root Vertex: Must be labelled by the Start Symbol

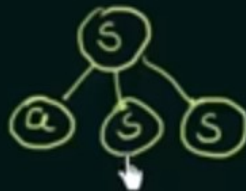
Vertex: Labelled by Non-Terminal Symbols

Leaves: Labelled by Terminal Symbols or ϵ



Left Derivation Tree

A Left Derivation Tree is obtained by applying production to the leftmost variable in each step.



Eg. For generating the string aabaa from the Grammar $S \rightarrow aAS \mid aSS \mid \epsilon$, $A \rightarrow SbA \mid ba$

Right Derivation Tree

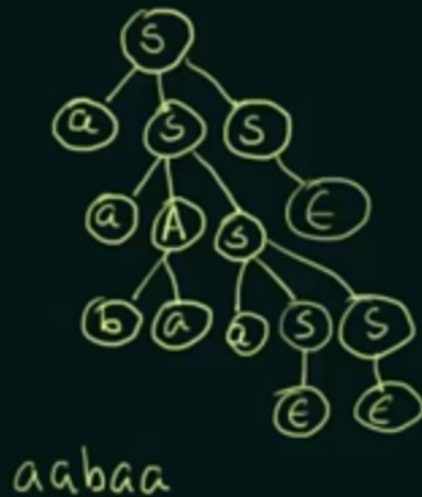
A Right Derivation Tree is obtained by applying production to the rightmost variable in each step.



Left Derivation Tree

A Left Derivation Tree is obtained by applying production to the leftmost variable in each step.

Eg. For generating the string aabaa from the Grammar $S \rightarrow aAS \mid aSS \mid \epsilon$, $A \rightarrow SbA \mid ba$



Right Derivation Tree

A Right Derivation Tree is obtained by applying production to the rightmost variable in each step.



Ambiguous Grammar

A Grammar is said to be Ambiguous if there exists two or more derivation tree for a string w (that means two or more left derivation trees)

Example: $G = (\{S\}, \{a+b, +, *\}, P, S)$ where P consists of $S \rightarrow S+S \mid S*S \mid a \mid b$

The String $a + a * b$ can be generated as:

$$\begin{aligned} S &\rightarrow S+S \\ &\rightarrow a+S \\ &\rightarrow a+S*S \\ &\rightarrow a+a*S \\ &\rightarrow a+a*b \end{aligned}$$

$$\begin{aligned} S &\rightarrow S*S \\ &\rightarrow S+S*S \\ &\rightarrow a+S*S \\ &\rightarrow a+a*S \\ &\rightarrow a+a*b \end{aligned}$$



Ambiguous Grammar

A Grammar is said to be Ambiguous if there exists two or more derivation tree for a string w (that means two or more left derivation trees)

Example: $G = (\{S\}, \{a+b, +, *\}, P, S)$ where P consists of $S \rightarrow S+S \mid S*S \mid a \mid b$

The String $a + a * b$ can be generated as:

$$S \rightarrow S + S$$

$$\rightarrow a + \underline{S}$$

$$\rightarrow a + S * S$$

$$\rightarrow a + a * S$$

$$\rightarrow a + a * b$$

$$S \rightarrow \underline{S} * S$$

$$\rightarrow \underline{S} + S * S$$

$$\rightarrow a + \underline{S} * S$$

$$\rightarrow a + a * \underline{S}$$

$$\rightarrow a + a * b$$

Thus, this Grammar is Ambiguous



Simplification of Context Free Grammar

Reduction of CFG

In CFG, sometimes all the production rules and symbols are not needed for the derivation of strings. Besides this, there may also be some NULL Productions and UNIT Productions. Elimination of these productions and symbols is called Simplification of CFG.

Simplification consists of the following steps:

- 1) Reduction of CFG
- 2) Removal of Unit Productions
- 3) Removal of Null Productions



Simplification of Context Free Grammar

Reduction of CFG

In CFG, sometimes all the production rules and symbols are not needed for the derivation of strings. Besides this, there may also be some NULL Productions and UNIT Productions. Elimination of these productions and symbols is called Simplification of CFG.

Simplification consists of the following steps:

- 1) Reduction of CFG
- 2) Removal of Unit Productions
- 3) Removal of Null Productions

REDUCTION OF CFG

CFG are reduced in two phases

Phase 1: Derivation of an equivalent grammar G' , from the CFG, G , such that each variable derives some terminal string

Derivation Procedure:

- Step 1: Include all Symbols W_1 , that derives some terminal and initialize $i = 1$
- Step 2: Include symbols W_{i+1} , that derives W_i
- Step 3: Increment i and repeat Step 2, until $W_{i+1} = W_i$
- Step 4: Include all production rules that have W_i in it



Simplification of Context Free Grammar

Reduction of CFG

In CFG, sometimes all the production rules and symbols are not needed for the derivation of strings. Besides this, there may also be some NULL Productions and UNIT Productions. Elimination of these productions and symbols is called Simplification of CFG.

Simplification consists of the following steps:

- 1) Reduction of CFG
- 2) Removal of Unit Productions
- 3) Removal of Null Productions

REDUCTION OF CFG

CFG are reduced in two phases

Phase 1: Derivation of an equivalent grammar G' , from the CFG, G , such that each variable derives some terminal string

Derivation Procedure:

- Step 1: Include all Symbols W_1 , that derives some terminal and initialize $i = 1$
- Step 2: Include symbols W_{i+1} , that derives W_i
- Step 3: Increment i and repeat Step 2, until $W_{i+1} = W_i$
- Step 4: Include all production rules that have W_i in it



REDUCTION OF CFG

CFG are reduced in two phases

Phase 1: Derivation of an equivalent grammar G' , from the CFG, G , such that each variable derives some terminal string

Derivation Procedure:

Step 1: Include all Symbols W_1 , that derives some terminal and initialize $i = 1$

Step 2: Include symbols W_{i+1} , that derives W_i

Step 3: Increment i and repeat Step 2, until $W_{i+1} = W_i$

Step 4: Include all production rules that have W_i in it

Phase 2: Derivation of an equivalent grammar G'' , from the CFG, G' , such that each symbol appears in a sentential form

Derivation Procedure:

Step 1: Include the Start Symbol in Y_1 and initialize $i = 1$

Step 2: Include all symbols Y_{i+1} , that can be derived from Y_i and include all production rules that have been applied

Step 3: Increment i and repeat Step 2, until $Y_{i+1} = Y_i$



Example: Find a reduced grammar equivalent to the grammar G , having production rules
P: $S \rightarrow AC|B$, $A \rightarrow a$, $C \rightarrow c|BC$, $E \rightarrow aA|e$



Phase 1

$$T = \{a, c, e\}$$

$$W_1 = \{A, C, E\}$$

$$W_2 = \{A, C, E, S\}$$

$$W_3 = \{A, C, E, S\}$$

$$Q' = \{(A, C, E, S), \{a, c, e\}, P, (S)\}$$

$$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c, E \rightarrow aA|e$$



$$W_2 = \{A, C, E, S\}$$

$$W_3 = \{A, C, E, S\}$$

$$G' = \{(A, C, E, S), \{a, c, e\}, P, (S)\}$$

$$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c, E \rightarrow aA|e$$

Phase 2 : $\gamma_1 = \{S\}$

$$\gamma_2 = \{S, A, C\}$$

$$\gamma_3 = \{S, A, C, a, c\}$$

$$\gamma_4 = \{S, A, C, a, c\}$$

$$G'' = \{(A, C, S), \{a, c\}, P, \{S\}\}$$



$$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c$$



Example: Find a reduced grammar equivalent to the grammar G , having production rules
 $P: S \rightarrow AC|B, A \rightarrow a, C \rightarrow c|BC, E \rightarrow aA|e$

Phase 1: $T = \{a, c, e\}$

$$W_1 = \{A, C, E\}$$

$$W_2 = \{A, C, E, S\}$$

$$W_3 = \{A, C, E, S\}$$

$$G' = \{(A, C, E, S), \{a, c, e\}, P, (S)\}$$

$$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c, E \rightarrow aA|e$$

Phase 2: $\gamma_1 = \{S\}$

$$\gamma_2 = \{S, A, C\}$$



Simplification of Context Free Grammar

Removal of Unit Productions

Any Production Rule of the form $A \rightarrow B$ where $A, B \in \text{Non Terminals}$ is called Unit Production

Procedure for Removal

Step 1: To remove $A \rightarrow B$, add production $A \rightarrow x$ to the grammar rule whenever $B \rightarrow x$ occurs in the grammar. [$x \in \text{Terminal}$, x can be Null]

Step 2: Delete $A \rightarrow B$ from the grammar.

Step 3: Repeat from Step 1 until all Unit Productions are removed.



Simplification of Context Free Grammar

Removal of Unit Productions

Any Production Rule of the form $A \rightarrow B$ where $A, B \in \text{Non Terminals}$ is called Unit Production

Procedure for Removal

Step 1: To remove $A \rightarrow B$, add production $A \rightarrow x$ to the grammar rule whenever $B \rightarrow x$ occurs in the grammar. [$x \in \text{Terminal}$, x can be Null]

Step 2: Delete $A \rightarrow B$ from the grammar.

Step 3: Repeat from Step 1 until all Unit Productions are removed.

Example: Remove Unit Productions from the Grammar whose production rule is given by

P: $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow Z|b$, $Z \rightarrow M$, $M \rightarrow N$, $N \rightarrow a$



Simplification of Context Free Grammar

Removal of Unit Productions

Any Production Rule of the form $A \rightarrow B$ where $A, B \in \text{Non Terminals}$ is called Unit Production

Procedure for Removal

Step 1: To remove $A \rightarrow B$, add production $A \rightarrow x$ to the grammar rule whenever $B \rightarrow x$ occurs in the grammar. [$x \in \text{Terminal}$, x can be Null]

Step 2: Delete $A \rightarrow B$ from the grammar.

Step 3: Repeat from Step 1 until all Unit Productions are removed.

Example: Remove Unit Productions from the Grammar whose production rule is given by

P: $S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow M, M \rightarrow N, N \rightarrow a$

$Y \rightarrow Z, Z \rightarrow M, M \rightarrow N$

1) Since $N \rightarrow a$, we add $M \rightarrow a$

P: $S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow M, M \rightarrow a, N \rightarrow a$



Step 2: Delete $A \rightarrow B$ from the grammar.

Step 3: Repeat from Step 1 until all Unit Productions are removed.

Example: Remove Unit Productions from the Grammar whose production rule is given by
P: $S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow M, M \rightarrow N, N \rightarrow a$

$Y \rightarrow Z, \quad \underline{Z \rightarrow M}, \quad \underline{M \rightarrow N}$

1) Since $N \rightarrow a$, we add $M \rightarrow a$

P: $S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow M, M \rightarrow a, N \rightarrow a$

2) Since $M \rightarrow a$, we add $Z \rightarrow a$

P: $S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$

3) Since $Z \rightarrow a$, we add $Y \rightarrow a$

P: $S \rightarrow XY, X \rightarrow a, Y \rightarrow a|b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$



$$\underline{Y \rightarrow Z}, \quad \underline{Z \rightarrow M}, \quad \underline{M \rightarrow N}$$

1) Since $N \rightarrow a$, we add $M \rightarrow a$

$$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow M, M \rightarrow a, N \rightarrow a$$

2) Since $M \rightarrow a$, we add $Z \rightarrow a$

$$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$$

3) Since $Z \rightarrow a$, we add $Y \rightarrow a$

$$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow a|b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$$

Remove the Unreachable Symbols

$$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow a|b$$



Simplification of Context Free Grammar

Removal of Null Productions

In a CFG, a Non-Terminal Symbol 'A' is a nullable variable if there is a production $A \rightarrow \epsilon$ or there is a derivation that starts at 'A' and leads to ϵ . (Like $A \rightarrow \dots \rightarrow \epsilon$)

Procedure for Removal:

- Step 1: To remove $A \rightarrow \epsilon$, look for all productions whose right side contains A
- Step2: Replace each occurrences of 'A' in each of these productions with ϵ
- Step 3: Add the resultant productions to the Grammar



Simplification of Context Free Grammar

Removal of Null Productions

In a CFG, a Non-Terminal Symbol 'A' is a nullable variable if there is a production $A \rightarrow \epsilon$ or there is a derivation that starts at 'A' and leads to ϵ . (Like $A \rightarrow \dots \rightarrow \epsilon$)

Procedure for Removal:

Step 1: To remove $A \rightarrow \epsilon$, look for all productions whose right side contains A

Step2: Replace each occurrences of 'A' in each of these productions with ϵ

Step 3: Add the resultant productions to the Grammar

Example: Remove Null Productions from the following Grammar

$S \rightarrow ABAC, A \rightarrow aA | \epsilon, B \rightarrow bB | \epsilon, C \rightarrow c$



or there is a derivation that starts at 'A' and leads to ϵ . (Like $A \rightarrow \dots \rightarrow \epsilon$)

Procedure for Removal:

Step 1: To remove $A \rightarrow \epsilon$, look for all productions whose right side contains A

Step 2: Replace each occurrence of 'A' in each of these productions with ϵ

Step 3: Add the resultant productions to the Grammar

Example: Remove Null Productions from the following Grammar

$S \rightarrow ABAC, A \rightarrow aA | \epsilon, B \rightarrow bB | \epsilon, C \rightarrow c$

$A \rightarrow \epsilon, B \rightarrow \epsilon$

1) To eliminate $A \rightarrow \epsilon$

$S \rightarrow \underline{A}B\underline{A}C$

$S \rightarrow ABC | BAC | BC$



Step2: Replace each occurrences of 'A' in each of these productions with ϵ

Step 3: Add the resultant productions to the Grammar

Example: Remove Null Productions from the following Grammar

$S \rightarrow ABAC, A \rightarrow aA | \epsilon, B \rightarrow bB | \epsilon, C \rightarrow c$

$A \rightarrow \epsilon, B \rightarrow \epsilon$

1) To eliminate $A \rightarrow \epsilon$

$S \rightarrow \underline{A}B\underline{A}C$

$S \rightarrow ABC | BAC | BC$

$A \rightarrow aA$

$A \rightarrow a$

New production: $S \rightarrow ABAC | ABC | BAC | BC$
 $A \rightarrow aA | a, B \rightarrow bB | \epsilon, C \rightarrow c$



1) Normal

$$S \rightarrow ABC | BAC | BC$$

$$A \rightarrow aA$$

$$A \rightarrow a$$

New production: $S \rightarrow ABAC | ABC | BAC | BC$
 $A \rightarrow aA | a, B \rightarrow bB | \epsilon, C \rightarrow c$

2) To eliminate $B \rightarrow \epsilon$

$$S \rightarrow AAC | AC | c, B \rightarrow b$$

New production: $S \rightarrow ABAC | ABC | BAC | BC | AAC | AC | c$
 $A \rightarrow aA | a$
 $B \rightarrow bB | b$
 $C \rightarrow c$



Chomsky's Normal Form (CNF)

CNF stands for chomsky normal form. A CFG is in CNF if all production rules satisfy one of the following conditions:-

- ★ Start symbol generating ϵ .

eg:- $A \rightarrow \epsilon$

- ★ A non terminal generating two non-terminals.

eg:- $S \rightarrow AB$

- ★ A non terminal generating a terminal.

CNF stands for chomsky normal form. A CFG is in CNF if all production rules satisfy one of the following conditions:-

★ Start symbol generating ϵ .

Eg:- $A \rightarrow \epsilon$ ✓

★ A non terminal generating two non-terminals.

Eg:- $S \rightarrow AB$

★ A non terminal generating a terminal.

Eg:- $S \rightarrow a$

For example 1-

$$G_1 = \{ \begin{array}{l} S \rightarrow AB, \\ S \rightarrow C \\ A \rightarrow a \\ B \rightarrow b \end{array} \}$$

G_1 satisfy the rule specified for CNF. So, it is in CNF

$$G_2 = \{ \begin{array}{l} S \rightarrow aA, \\ A \rightarrow a \\ B \rightarrow c \end{array} \}$$

G_2 does not satisfy the rules specified for CNF as $S \rightarrow aA$ contains terminal followed by non-terminal
So G_2 is not in CNF.

for example 1-

$$G_1 = \left\{ \begin{array}{l} S \rightarrow AB, \\ S \rightarrow c, \\ A \rightarrow a, \\ B \rightarrow b \end{array} \right\}$$

G_1 satisfy the rule specified for CNF. so, it is in CNF

$A \rightarrow \epsilon$
 $S \rightarrow AB$
 $A \rightarrow a$

$$G_2 = \left\{ \begin{array}{l} S \rightarrow aA, \\ A \rightarrow a, \\ B \rightarrow c \end{array} \right\}$$

G_2 does not satisfy the rules specified for CNF as $S \rightarrow aA$ contains terminal followed by non-terminal so G_2 is not in CNF.

Steps for converting CFG into CNF

Step 1 :- Eliminate start symbol from the RHS.

If the start symbol T is at the right-hand side of any production, Create a production as :-

$$S_1 \rightarrow T$$

where S_1 is the new start symbol.

Step 2 :- In the grammar, remove the null, unit and useless productions.

Step 3 :- Eliminate terminals from the RHS of the production if they exist with other non-terminals or terminals.

Steps for converting CFG into CNF

$S \rightarrow \epsilon$
 $A \rightarrow SA \mid AB$
 $A \rightarrow a$

Step 1: Eliminate start symbol from the RHS.

If the start symbol S is at the right-hand side of any production, create a production as:-

$$S_1 \rightarrow S$$

where S_1 is the new start symbol.

Step 2: In the grammar, remove the null, unit and useless productions.

Step 3: Eliminate terminals from the RHS of the production if they exist with other non-terminals or terminals.

Step 1 :- Eliminate start symbol from the RHS.

If the start symbol T is at the right-hand side of any production, Create a production as:-

$$\textcircled{S} \rightarrow S$$

where S is the new start symbol.

Step 2 :- In the grammar, remove the null, unit and useless productions.

Step 3 :- Eliminate terminals from the RHS of the production if they exists with other non-terminals or terminals.

eg:- $\textcircled{S \rightarrow aA}$ can be decomposed as:

$$A \rightarrow a$$

S

if is
in CNF

$$\left\{ \begin{array}{l} S \rightarrow RA \\ R \rightarrow a \end{array} \right\}$$

$$\Rightarrow \underline{S \rightarrow aA} \Rightarrow \text{decomposed.}$$

Step 4 : Eliminate RHS with more than two non-terminals.

eg: $S \rightarrow \underline{A} \underline{B} S$ can be decomposed as :

CNF

$$\left\{ \begin{array}{l} S \rightarrow RS \\ R \rightarrow AB \end{array} \right\}$$

$$S \rightarrow \underline{AB} S$$

$$S \rightarrow \underline{AB} S$$

$\downarrow$
 $S \rightarrow R$

Pumping Lemma (For Regular Languages)

>> Pumping Lemma is used to prove that a Language is NOT REGULAR

>> It cannot be used to prove that a Language is Regular

If A is a Regular Language, then A has a Pumping Length ' P ' such that any string ' S ' where $|S| \geq P$ may be divided into 3 parts $S = x y z$ such that the following conditions must be true:

- (1) $x y^i z \in A$ for every $i \geq 0$
- (2) $|y| > 0$
- (3) $|xy| \leq P$



$$(2) |y| > 0$$

$$(3) |xy| \leq P$$

To prove that a language is not Regular using PUMPING LEMMA, follow the below steps:

(We prove using Contradiction)

- > Assume that A is Regular
- > It has to have a Pumping Length (say P)
- > All strings longer than P can be pumped $|S| \geq P$
- > Now find a string 'S' in A such that $|S| \geq P$
- > Divide S into x y z
- > Show that $x y^i z \notin A$ for some i
- > Then consider all ways that S can be divided into x y z
- > Show that none of these can satisfy all the 3 pumping conditions at the same time
- > S cannot be Pumped == CONTRADICTION



Pumping Lemma (for Context Free Languages)

Pumping lemma (for CFL) is used to prove that a language is not a context free.

Assume L is a context free lang. there is a pumping length n such that any string $w \in L$ of length $\geq n$ can be written as

$$|w| \geq n$$

We can break w into 5 strings, $w = uvxyz$, such that

$$\rightarrow |vxy| \leq n$$

$$\rightarrow |vy| \neq \epsilon$$

Pumping lemma (for Context Free Languages)

Pumping lemma (for CFL) is used to prove that a language is not a Context free.

CFG
↓
4 tuples, $\{V, E\}$

Assume L is a context free lang., there is a pumping length n such that any string $w \in L$ of length $\geq n$ can be written as

$$|w| \geq n$$

We can break w into 5 strings, $w = uvxyz$

$$\rightarrow |vxy| \leq n$$

$$\rightarrow |vy| \neq \epsilon$$

Pumping lemma (for Context Free Languages)

Pumping lemma (for CFL) is used to prove that a language is not a Context free.

CFG
↓
4 tuples. YES

~~indicates~~ Assume L is a context free lang. there is a pumping length n such that any string $w \in L$ of length $\geq n$ can be written as

$$|w| \geq n$$

①

We can break w into 5 strings, $w = uvxyz$, such that

$$\rightarrow |vxy| \leq n$$

$$\rightarrow |vy| \neq \epsilon$$

Pumping lemma (for CFL) is used to prove that a CFG language is not a Context free.

$\Downarrow$
 $4 \text{ tuples } \{V, E, S, P\}$

Assume L is a context free lang. there is a pumping length n such that any string $w \in L$ of length $\geq n$ can be written as

$$|w| \geq n$$

We can break w into 5 strings, $w = uvxyz$, such that

$$\rightarrow |vxy| \leq n$$

$$\rightarrow |vy| \neq \epsilon$$

$\rightarrow$ For all $k \geq 0$, the string $uv^kxv^kyz \in L$.

Note: select k such that the resulting string is not in L .

(CFL) follow the steps:- (We prove using contradiction)

- 1) Assume that L is context free
- 2) The pumping length say n
- 3) All strings longer than n can be pumped. $|w| \geq n$
- 4) Now find a string ' w ' in L such that $|w| \geq n$.
- 5) Divide w into $uvxyz$
- 6) Show that ~~$u^k v^k x y^k z$~~ $u v^k x y^k z \notin L$ for some k .
- 7) Then consider the ways that w can be divided into $uvxyz$
- 8) Show that none of these can satisfy all the 3 pumping conditions at same time.
- 9) w cannot be pumped (contradiction)

Example 1 Find out whether $L = \{x^n y^n z^n \mid n \geq 1\}$ is context free or not

Solution 1 Let L is context free

$\Rightarrow$ now we can take a string such that
$$S = x^n y^n z^n$$

$\Rightarrow$ we divide S in 5 parts $uvxyz$

Context free or not

Question

Let L is context free

$\Rightarrow$ now we can take a string such that

$$S = x^n y^n z^n$$

$\Rightarrow$ we divide S in 5 parts $uvxzy$

Case (i) - $n=4$ so, $S = x^4 y^4 z^4$

$\forall \epsilon > 0$ Each contain only one type of symbols.

$x x x x y y y y z z z z$
↓ ↓ ↓ ↓
 $x x y z$

$\Rightarrow$ divide into 5 parts
 $uvxzy$

$$u v^k x y^k z \quad (\text{let take } k=2)$$

$$\Rightarrow u v^2 x y^2 z$$

$$\Rightarrow x x x x x y y y y z z z z z$$

$$\Rightarrow x^6 y^4 z^5$$

$$\Rightarrow x^6 y^4 z^5 \notin L$$

$$u = x$$

$$v = x x$$

$$x = x y y y y z$$

$$y = z$$

$$z = z z$$

$n=4$, So $S = x^4 y^4 z^4$

$\Rightarrow \cancel{uv^k x y^k z} \notin L$

$\Rightarrow x x x x y y y y z z z z \Rightarrow 5 \text{ parts}$
 $\downarrow$
 $u \quad v \quad x \quad y \quad z$

$\Rightarrow$ let assume $k=2$.

$\Rightarrow uv^k x y^k z$

$\Rightarrow uv^2 x y^2 z$

$\Rightarrow x x x x y y x x y y y y z z z z$

$\Rightarrow \underline{x^4 y^2 x^2 y^5 z^4} \notin L$

of our string $x^n y^n z^n$

$u = xx$

$v = xxyy$

$x = \underline{y}$

$y = \underline{y}$

$z = zzzz$

Pushdown Automata (Introduction)

A Pushdown Automata (PDA) is a way to implement a Context Free Grammar in a similar way we design Finite Automata for Regular Grammar

- > It is more powerful than FSM
- > FSM has a very limited memory but PDA has more memory
- > PDA = Finite State Machine + A Stack

A stack is a way we arrange elements one on top of another

A stack does two basic operations:

PUSH: A new element is added at the Top of the stack

POP: The Top element of the stack is read and removed



Pushdown Automata (Introduction)

A Pushdown Automata (PDA) is a way to implement a Context Free Grammar in a similar way we design Finite Automata for Regular Grammar

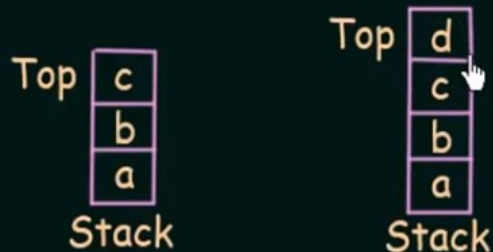
- > It is more powerful than FSM
- > FSM has a very limited memory but PDA has more memory
- > PDA = Finite State Machine + A Stack

A stack is a way we arrange elements one on top of another

A stack does two basic operations:

PUSH: A new element is added at the Top of the stack

POP: The Top element of the stack is read and removed



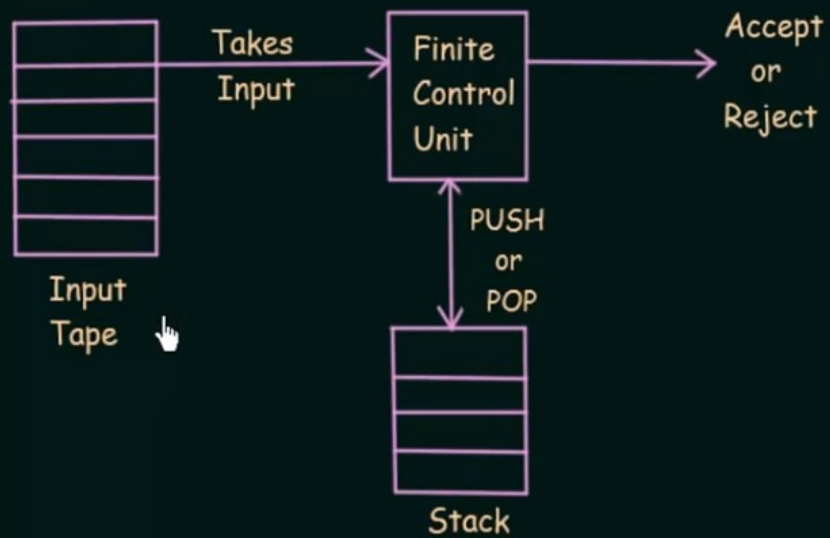
A Pushdown Automata has 3 components:

- 1) An input tape
- 2) A Finite Control Unit
- 3) A Stack with infinite size



A Pushdown Automata has 3 components:

- 1) An input tape
- 2) A Finite Control Unit
- 3) A Stack with infinite size



Pushdown Automata (Formal Definition)

A Pushdown Automata is formally defined by 7 Tuple as shown below:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

where,

Q = A finite set of States

Σ = A finite set of Input Symbols

Γ = A finite Stack Alphabet

δ = The Transition Function

q_0 = The Start State

z_0 = The Start Stack Symbol

F = The set of Final / Accepting States



Pushdown Automata (Formal Definition)

A Pushdown Automata is formally defined by 7 Tuples as shown below:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

where,

Q = A finite set of States

Σ = A finite set of Input Symbols

Γ = A finite Stack Alphabet

δ = The Transition Function

q_0 = The Start State

z_0 = The Start Stack Symbol

F = The set of Final / Accepting States

δ takes as argument a triple $\delta(q, a, X)$ where:

- (i) q is a State in Q
- (ii) a is either an Input Symbol in Σ or $a = \epsilon$
- (iii) X is a Stack Symbol, that is a member of Γ



δ = The Transition Function

q_0 = The Start State

z_0 = The Start Stack Symbol

F = The set of Final / Accepting States

δ takes as argument a triple $\delta(q, a, X)$ where:

(i) q is a State in Q

(ii) a is either an Input Symbol in Σ or $a = \epsilon$

(iii) X is a Stack Symbol, that is a member of Γ

The output of δ is finite set of pairs (p, γ) where:

p is a new state

γ is a string of stack symbols that replaces X at the top of the stack

Eg. If $\gamma = \epsilon$ then the stack is popped

If $\gamma = X$ then the stack is unchanged

If $\gamma = YZ$ then X is replaced by Z and Y is pushed onto the stack

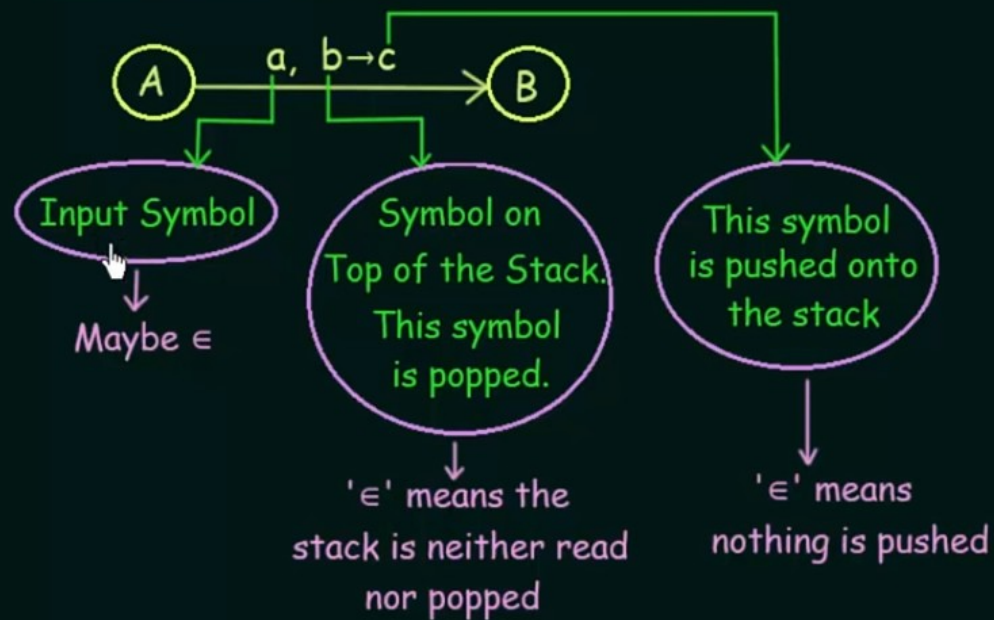


Pushdown Automata (Graphical Notation)

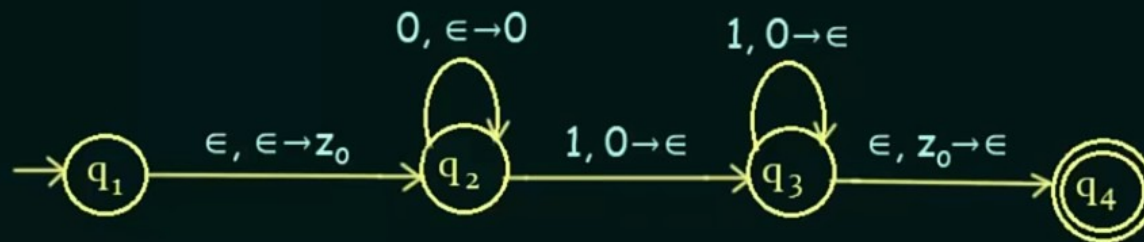
Finite State Machine



Pushdown Automata



Example: Construct a PDA that accepts $L = \{0^n 1^n \mid n \geq 0\}$



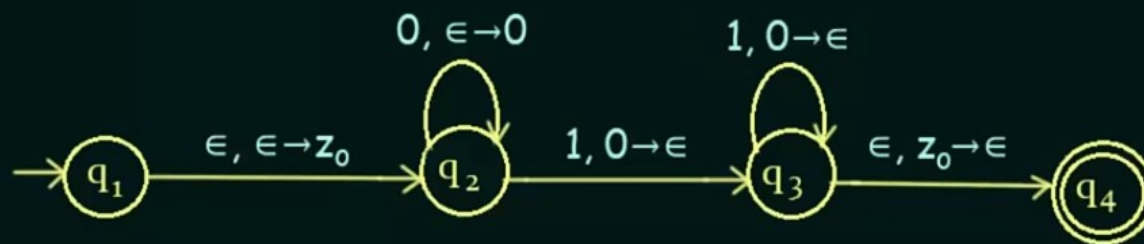
\$



00 11



Example: Construct a PDA that accepts $L = \{0^n 1^n \mid n \geq 0\}$



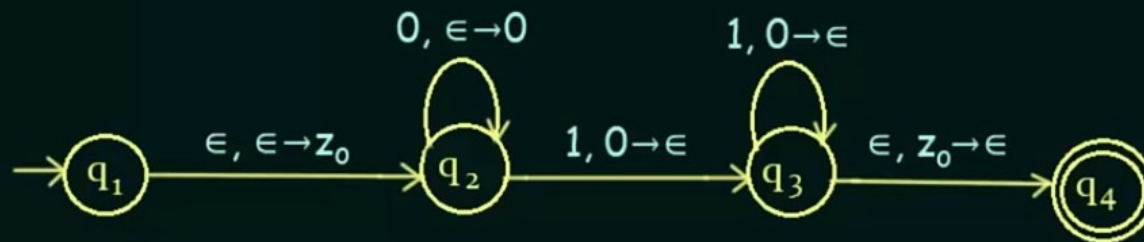
\$



00 11 ✓



Example: Construct a PDA that accepts $L = \{0^n 1^n \mid n \geq 0\}$



\$



00 11 ✓



unambiguous Grammar

→ A grammar can be unambiguous if the grammar does not contain ambiguity that means if it does not contain more than one leftmost derivation & more than one rightmost derivation & more than one parse tree for the given i/p string.

To convert ambiguous grammar to unambiguous grammar, we apply the following rules:-

- ① If the left associative operators (+, -, *, /) are used in the production rule, then apply left recursion in the production rule. Left recursion means that the leftmost symbol on the right side

→ A grammar can be unambiguous if the grammar does not contain ambiguity that means if it does not contain more than one leftmost derivation or more than one rightmost derivation or more than one parse tree for the given i/p string.

To convert ambiguous grammar to unambiguous grammar we apply the following rules:-

$S \rightarrow (A) + B * S /$

- ① If the left associative operators $(+, -, *, /)$ are used in the production rule, then apply left recursion in the production rule. Left recursion means that the leftmost symbol on the right side is the same as the non-terminal on the left side.

$$\textcircled{X} \rightarrow \textcircled{X}a$$

② If the right associative operator (r) is used in the production rule then apply right recursion in the production rule. Right recursion means that the rightmost symbol on the left side is the same as the non-terminal on the right side.

$$\text{Eg: } X \rightarrow aX.$$

$$\textcircled{X} \rightarrow \textcircled{X}a$$

If the right associative operator ($\wedge$) is used in the production rule then apply right recursion in the production rule.

Right recursion means that the rightmost symbol on the left side is the same as the non-terminal on the right side.

Eg: $\textcircled{X} \rightarrow a\textcircled{X}$

left associative operators (+, -, *, /)
right " " $\textcircled{\wedge}$



Example 1 Consider a grammar G is

$$S \rightarrow AB | aAB$$

$$A \rightarrow a | Aa$$

$$B \rightarrow b$$

Determine whether the grammar G is ambiguous or not.
If G is ambiguous, construct an unambiguous grammar equivalent to G .

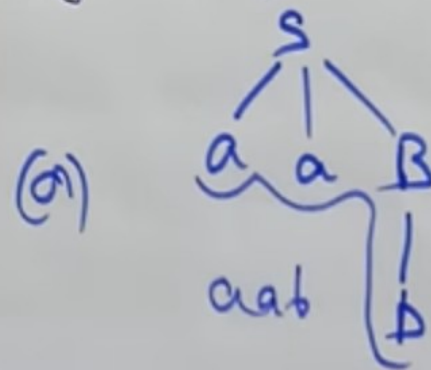
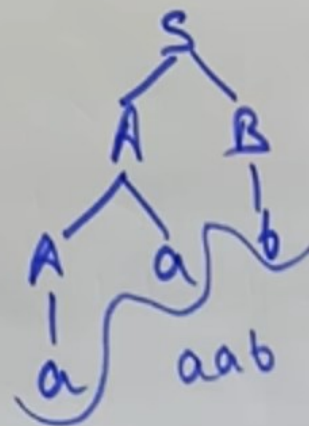
Solution 1.

$$\left. \begin{array}{l} S \rightarrow \underline{AB} | aaB \\ A \rightarrow a | \underline{Aa} \\ B \rightarrow b \end{array} \right\} \approx \text{unambiguous}$$

Determine whether the grammar G is ambiguous or not.
If G is ambiguous, construct an unambiguous grammar equivalent to G . if

Solution 1.

$\Rightarrow$ "aab"



① Removal of Useless Symbols :

A symbol can be useless if it does not appear on the right hand side of production & it does not take part in the derivation of any string. That symbol is useless.

eg: $T \rightarrow aab / abA / aaT$
 $A \rightarrow aA$
 $B \rightarrow ab / b$
 $C \rightarrow ad$

→ Here variable 'c' is useless b/c it is never occur in derivation string. so, we can eliminate.

first find all the variables which will never lead to a terminal string such ~~that~~ as var 'A'. Then we will remove all the productions in which var 'B' occurs.

$T \rightarrow aaB / abA / aaT$
 $\times \textcircled{A \rightarrow aA}$
 $B \rightarrow ab / \underline{b}$
 $C \rightarrow ad \times$

$T \rightarrow aaB$
 $\rightarrow \underline{aaab} / \underline{aab}$
 $T \rightarrow abA$
 $\rightarrow aa\underline{aA}$
 $\rightarrow \underline{aaaaA}$
 $\Rightarrow \underline{aaaaaA}$

To remove this useless production $A \Rightarrow aA$, ~~if~~ we will first find all the variables which will never lead to a terminal string such ~~that~~ as var 'A'. Then we will remove all the productions in which var 'B' occurs.

Push down Automata

A push down automaton is a way to implement a context-free grammar in a similar way we design DFA for a R.G.

- ⇒ A DFA can remember a finite amount of information but
- ⇒ A PDA can remember an infinite amount of information.

Basically a PDA is -

"finite state machine" + "a stack"

⇒ PDA has three components -

- ★ an i/p tape

- ★ a control unit and

A push down automaton is a way to implement a Context-free grammar in a similar way we design DFA for a R.G.

⇒ A DFA can remember a finite amount of information but

⇒ A PDA can remember an infinite amount of information. *

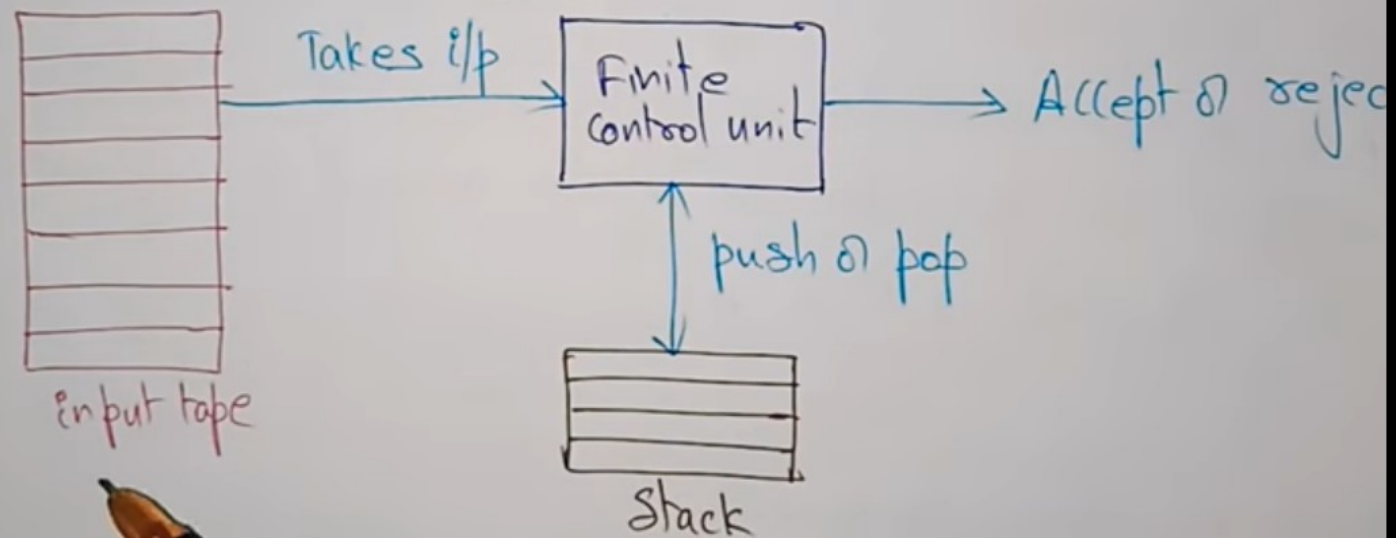
Basically a PDA is -

"finite state machine" + "a stack"

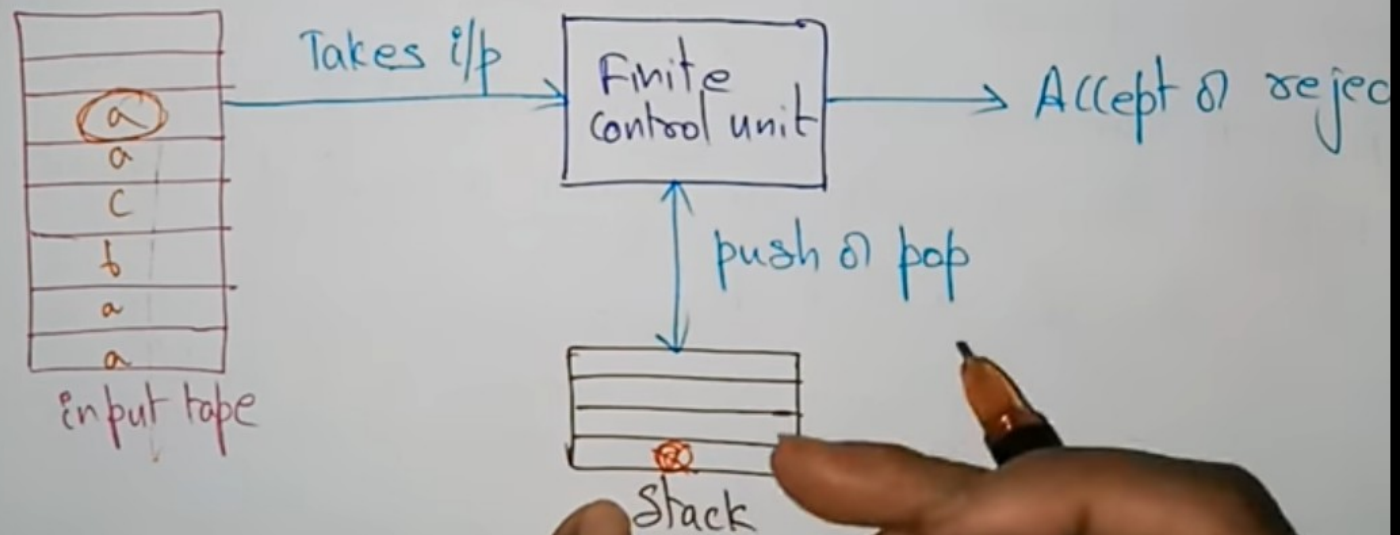
⇒ PDA has three components -

- * an i/p tape
- * a control unit and
- * a stack with infinite size.

→ APDA may or may not read i/p symbol, but it has to read the top of the stack in every transaction.



→ APDA may or may not read i/p symbol, but it has to read the top of the stack in every transaction.



A PDA can be formally described as 7-tuples $(Q, \Sigma, S, \delta, q_0, I, F)$

→ Q is finite no. of states

→ Σ is i/p alphabet

→ S is stack symbol

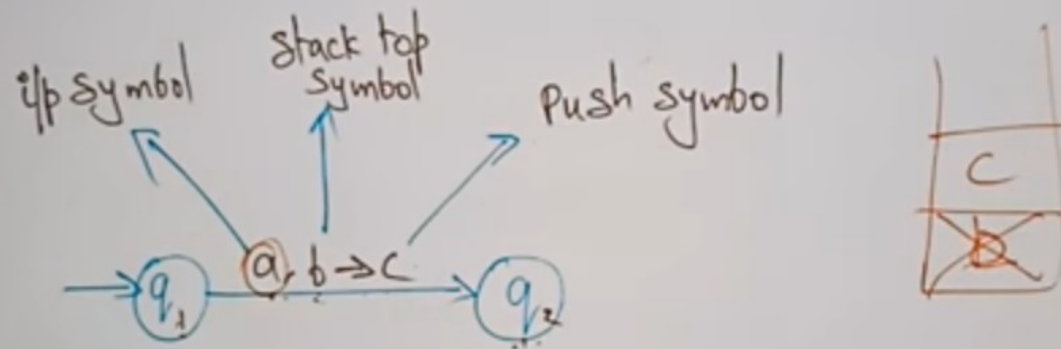
→ δ is the transition function: $Q \times (\Sigma \cup \{\epsilon\}) \times S \times \delta$

→ q_0 is the initial state ($q_0 \in Q$)

→ I is the initial stack top symbol ($I \in S$)

→ F is a set of accepting states ($F \subseteq Q$)

⇒ The dig that shows a transition in a PDA from a state q_1 to state q_2 , labeled as $a, b \rightarrow c$ -



i.e. at state q_1 , if an i/p string 'a' is encountered and top symbol of stack is 'b' then we pop 'b', push 'c' on top of the stack and move to state q_2 .

PDA can be formally described as 7-tuples $(Q, \Sigma, \underline{S}, \delta, q_0, I, F)$.

finite state m/c
+
stack

$\rightarrow Q$ is finite no. of states $q_0, q_1, q_2 \dots$

$\rightarrow \Sigma$ is i/p alphabet

$\rightarrow S$ is stack symbol

$\rightarrow \delta$ is the transition function: $Q \times (\Sigma \cup \{\epsilon\}) \times S \times Q \times S^*$

$\rightarrow q_0$ is the initial state ($q_0 \in Q$)

$\rightarrow I$ is the initial stack top symbol ($I \in S$)

$\rightarrow F$ is a set of accepting states ($F \subseteq Q$)

Terminologies Related to PDA

① Instantaneous Description (ID)

The instantaneous description (ID) of a PDA is represented by a triple (q, w, s) where

→ q is the state

→ w is unconsumed i/p

→ s is the stack contents.

ID = triple

→ Turnstile Notation

It is a notation of how a PDA computes on its string and make that string is accepted.

Terminologies Related to PDA

① Instantaneous Description (ID)

The instantaneous description (ID) of a PDA is represented by a triple (q, w, s) where

→ q is the state

→ w is unconsumed i/p

→ s is the stack contents

ID = triple

→ Informal Notation

ID is an informal notation of configuration. It represents the current state of the PDA, the input string, and the stack contents. The PDA can read the input string and make a decision based on the current state and the input string.

① Instantaneous Description (ID)

The instantaneous description (ID) of a PDA is represented by a triple (q, w, s) where

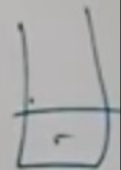
→ q is the state

→ w is unconsumed i/p

→ s is the stack contents

abba

$ID = \text{triple}$
 $ID = (q, w, s)$
↓ ↓
state stack
 q_0
 q_1



→ Informal Notation

ID is an informal notation of how a PDA computes an i/p string and make a decision that string is accepted or rejected.

→ Turnstile Notation :-

It is used for connecting pairs of ID's that represent one or ~~not~~ many moves of a PDA. The process of transition is denoted by the turnstile &

Symbol "⊢".

⊢ It represents one move

⊢* sign describes a sequence of moves

Example:- $(P, b, T) \vdash (q, w, a)$

While taking a transition from P to q, the i/p symbol 'b' is consumed, & top of the stack 'T' is represented

Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

Examples: NOON

NO LEMON NO MELON

123321

abba

RACECAR



Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

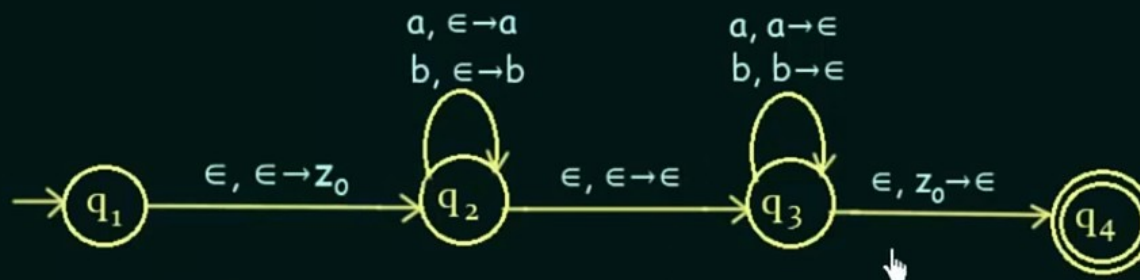
Examples: NOON

NO LEMON NO MELON

123321

abba

RACECAR



Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

Examples: NOON

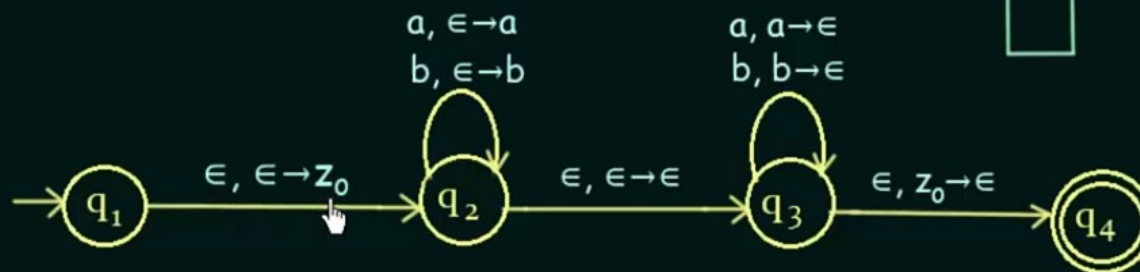
NO LEMON NO MELON

123321

abba

RACECAR

abba



Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

Examples: NOON

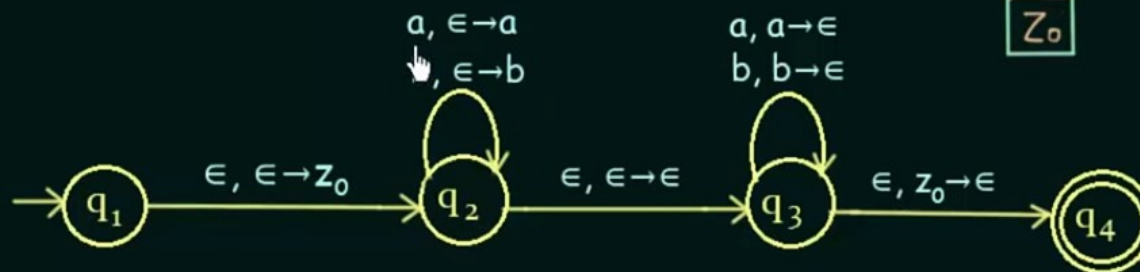
NO LEMON NO MELON

123321

abba

RACECAR

a b b a



Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

Examples: NOON

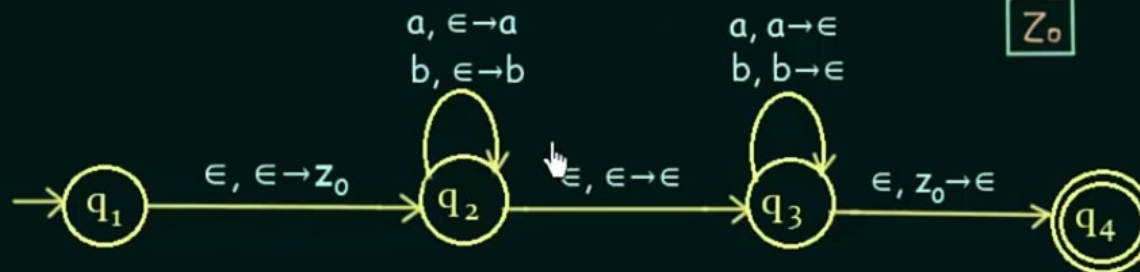
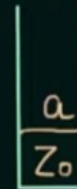
NO LEMON NO MELON

123321

abba

RACECAR

a b b a



Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

Examples: NOON

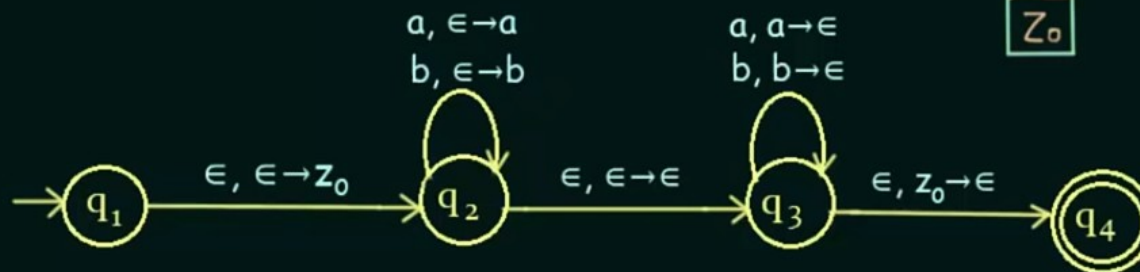
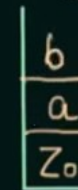
NO LEMON NO MELON

123321

abba

RACECAR

a b b a



Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

Examples: NOON

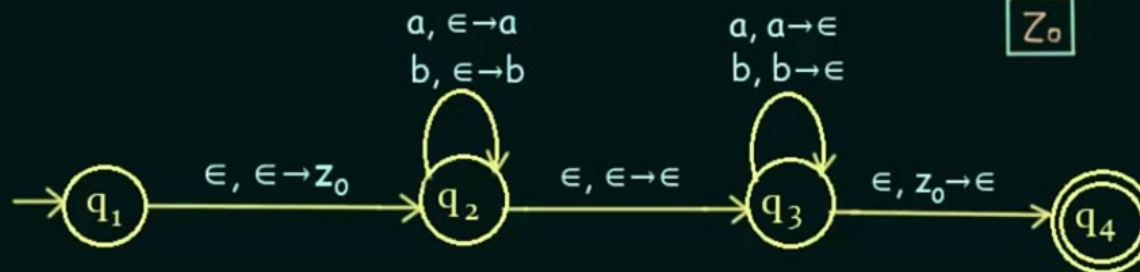
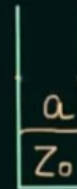
NO LEMON NO MELON

123321

abba

RACECAR

a b | b a



Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

Examples: NOON

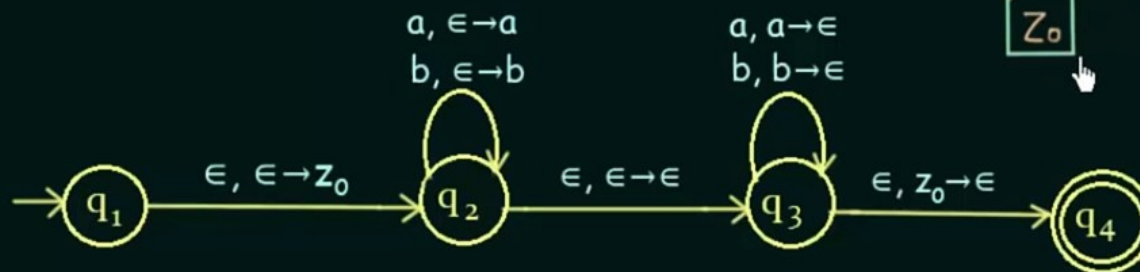
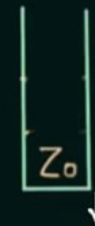
NO LEMON NO MELON

123321

abba

RACECAR

a b | b a



Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

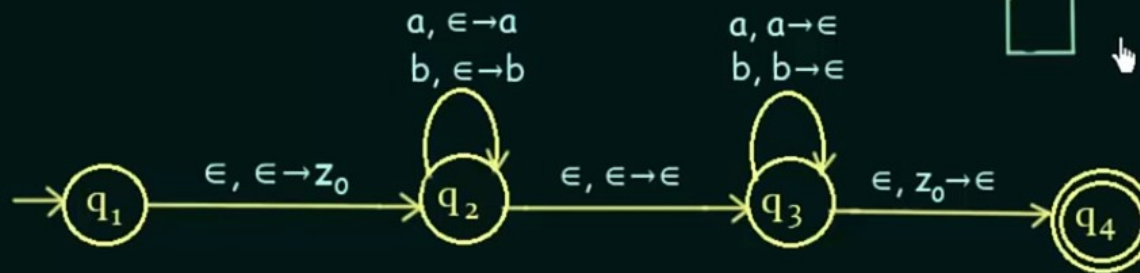
Examples: NOON

NO LEMON NO MELON

123321

abba

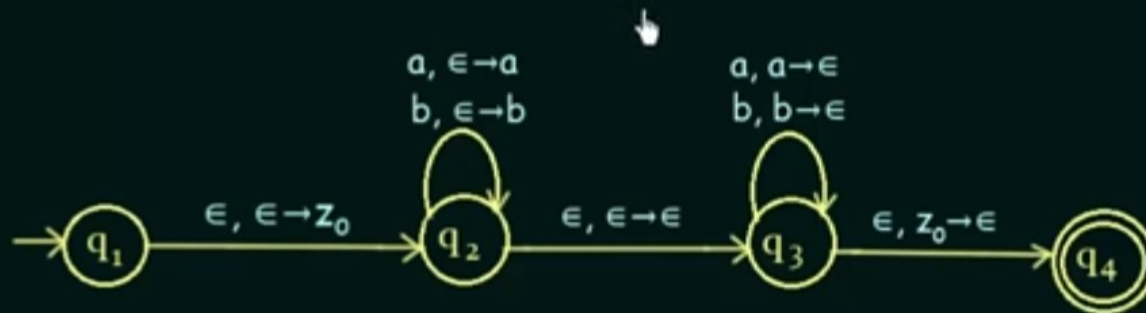
RACECAR



Pushdown Automata - Example (Even Palindrome) PART- 2

Construct a PDA that accepts Even Palindromes of the form

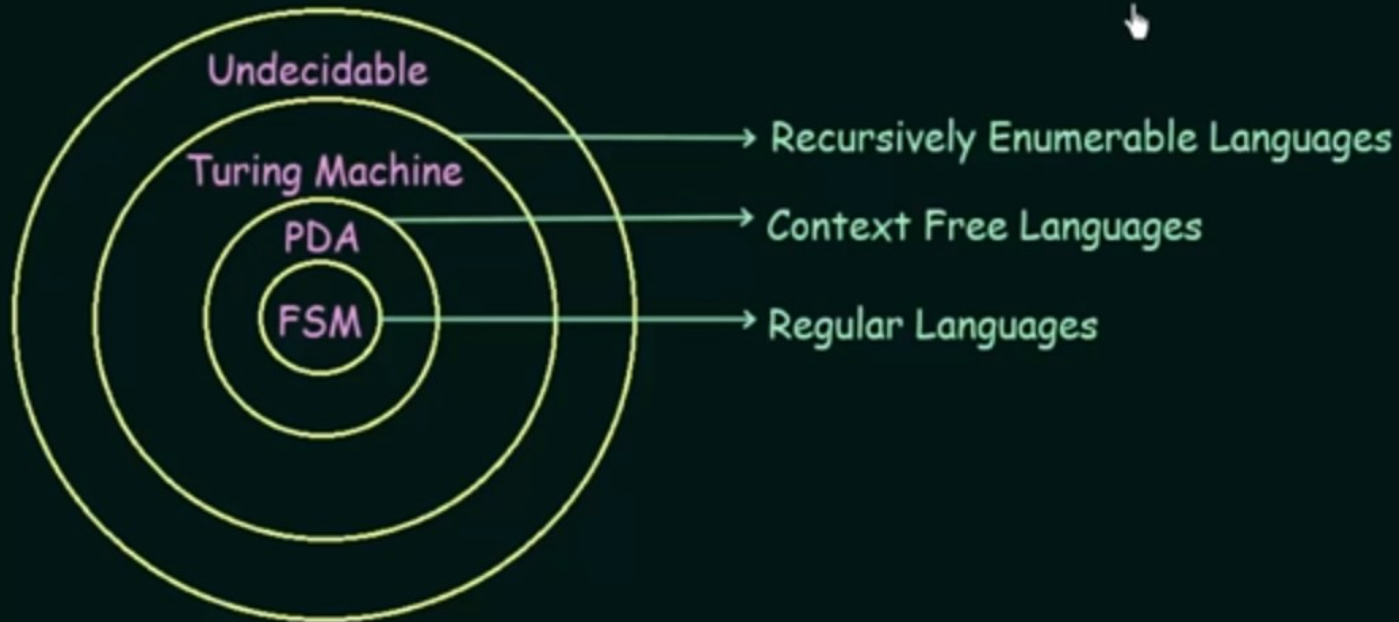
$$L = \{ ww^R \mid w = (a+b)^+ \}$$



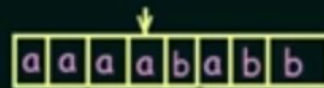
Example: a b b a



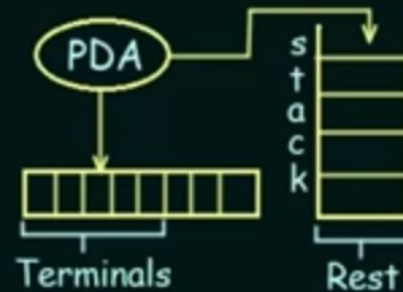
Turing Machine - Introduction (Part-1)



FSM: The Input String

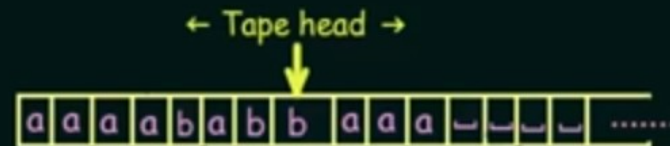


PDA: -> The Input String
-> A Stack



TURING MACHINE:

-> A Tape



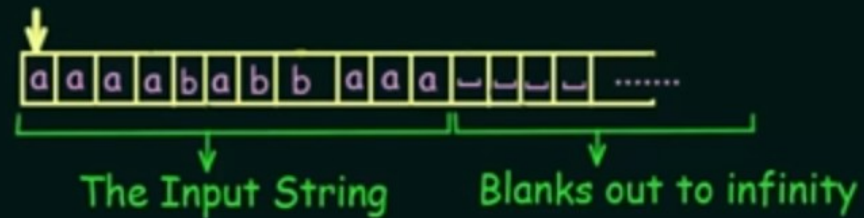
Tape Alphabets: $\Sigma = \{0, 1, a, b, x, Z_0\}$

The Blank $\sqcup$ is a special symbol. $\sqcup \notin \Sigma$

The blank is a special symbol used to fill the infinite tape



Initial Configuration:

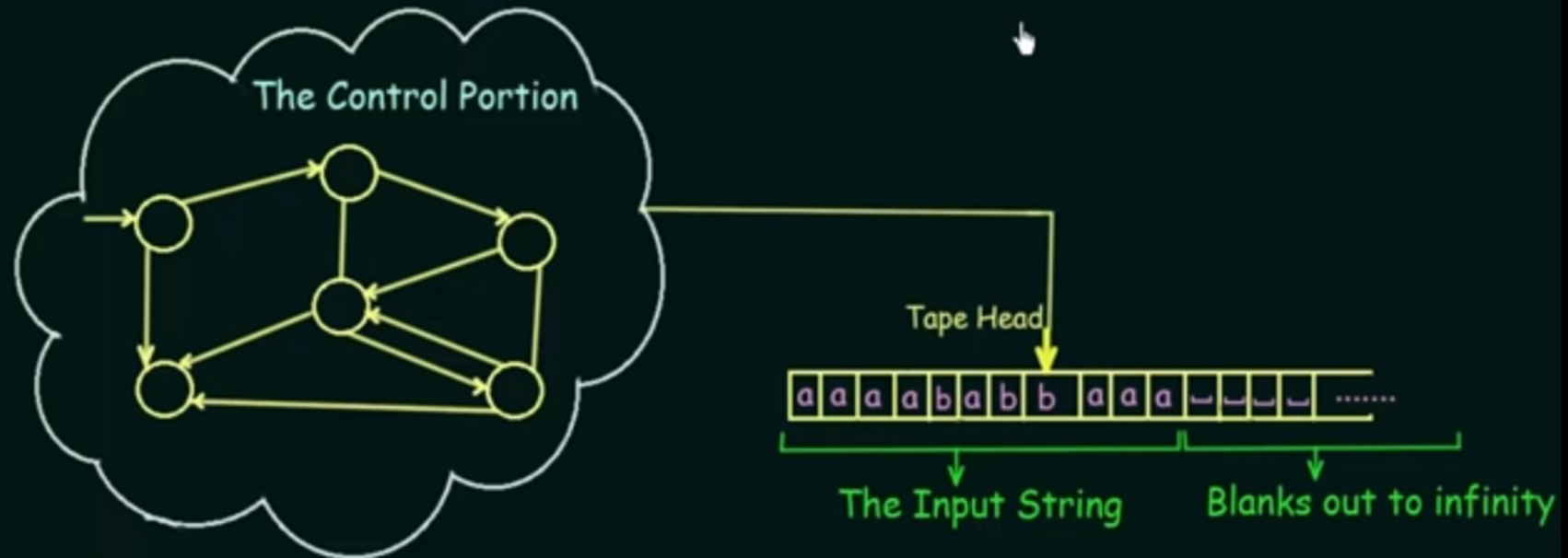


Operations on the Tape:

- > Read / Scan symbol below the Tape Head
- > Update / Write a symbol below the Tape Head
- > Move the Tape Head one step LEFT
- > Move the Tape Head one step RIGHT



Turing Machine - Introduction (Part-2)



The Control Portion similar to FSM or PDA

The PROGRAM

It is deterministic

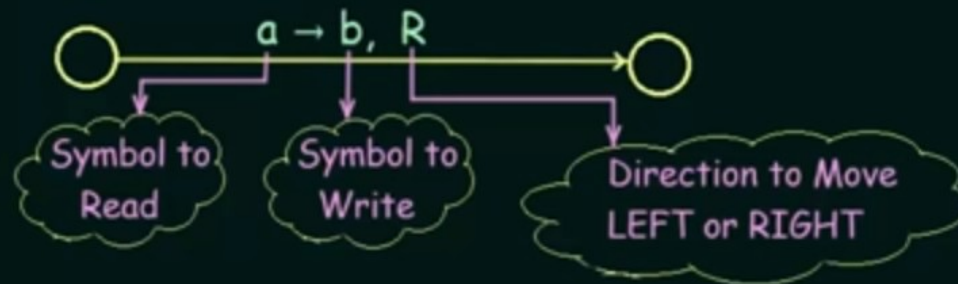


Rules of Operation - 1

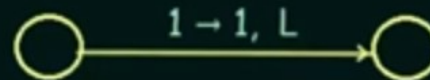
At each step of the computation:

- > Read the current symbol
- > Update (i.e. write) the same cell
- > Move exactly one cell either LEFT or RIGHT

If we are at the left end of the tape, and trying to move LEFT, then do not move.
Stay at the left end



If you don't want to update the cell,
JUST WRITE THE SAME SYMBOL



Rules of Operation - 2

- > Control is with a sort of FSM
- > Initial State
- > Final States: (there are two final states)
 - 1) The ACCEPT STATE
 - 2) The REJECT STATE
- > Computation can either
 - 1) HALT and ACCEPT
 - 2) HALT and REJECT
 - 3) LOOP (the machine fails to HALT)



Turing Machine (Formal Definition)

A Turing Machine can be defined as a set of 7 tuples

$$(Q, \Sigma, \Gamma, \delta, q_0, b, F)$$

$Q \rightarrow$ Non empty set of States

$\Sigma \rightarrow$ Non empty set of Symbols

$\Gamma \rightarrow$ Non empty set of Tape Symbols

$\delta \rightarrow$ Transition function defined as

$$Q \times \Sigma \rightarrow \Gamma \times (R/L) \times Q$$

$q_0 \rightarrow$ Initial State

$b \rightarrow$ Blank Symbol

$F \rightarrow$ Set of Final states (Accept state & Reject State)

Thus, the Production rule of Turing Machine will be written as

$$\delta(q_0, a) \rightarrow (q_1, \gamma, R)$$



Turing's Thesis:

Turing's Thesis states that any computation that can be carried out by mechanical means can be performed by some Turing Machine.

Few arguments for accepting this thesis are:

- i. Anything that can be done on existing digital computer can also be done by Turing Machine.
- ii. No one has yet been able to suggest a problem solvable by what we consider an algorithm, for which a Turing Machine Program cannot be written.



Turing's Thesis:

Turing's Thesis states that any computation that can be carried out by mechanical means can be performed by some Turing Machine.

Few arguments for accepting this thesis are:

- i. Anything that can be done on existing digital computer can also be done by Turing Machine.
- ii. No one has yet been able to suggest a problem solvable by what we consider an algorithm, for which a Turing Machine Program cannot be written.

Recursively Enumerable Language:

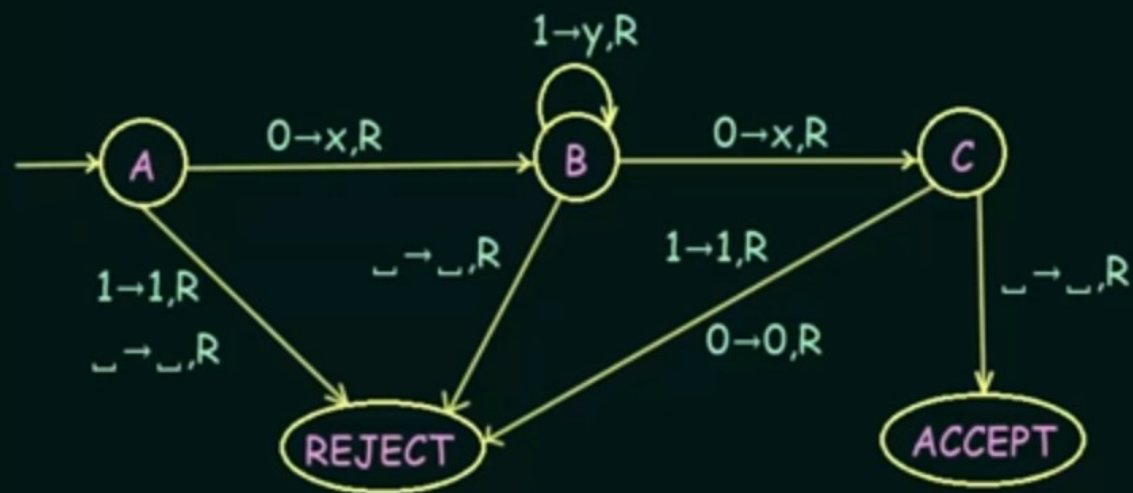
A Language L and Σ is said to be Recursively Enumerable if there exists a Turing Machine that accepts it.



Turing Machine - Example (Part-1)

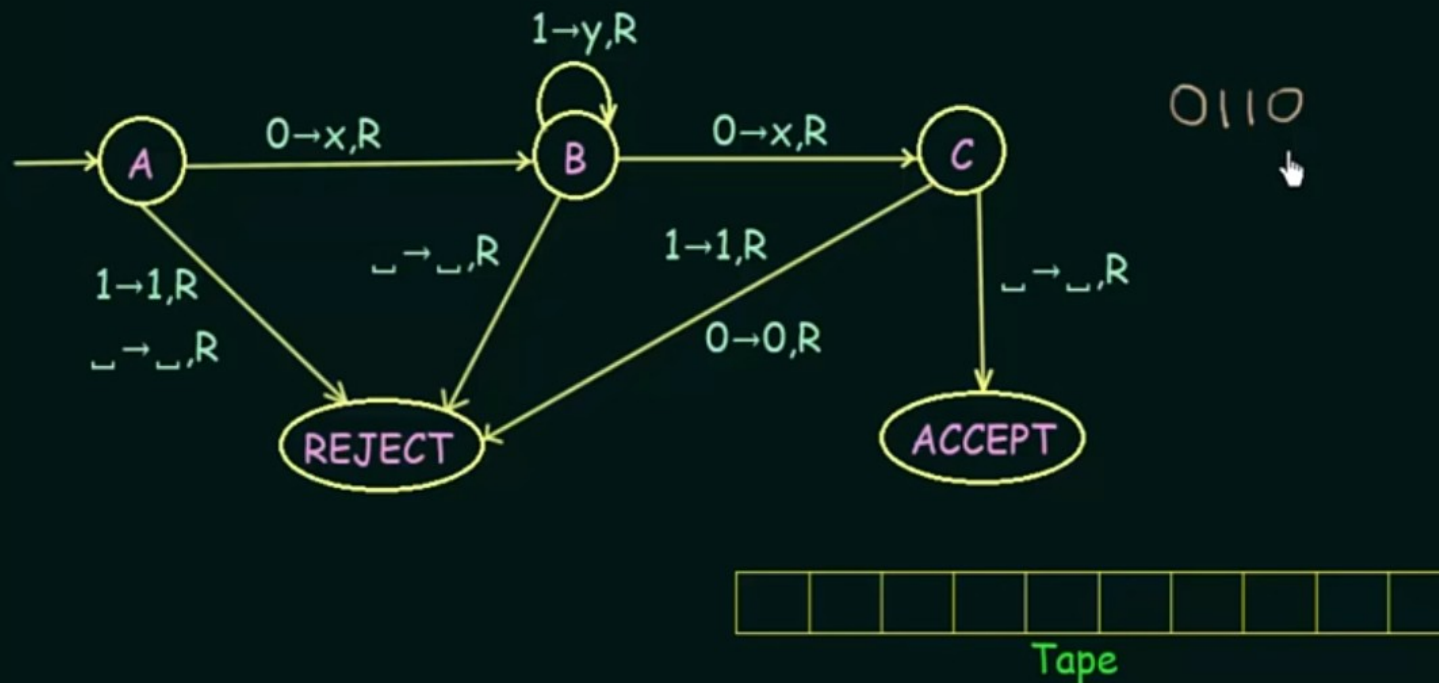
Design a Turing Machine which recognizes the language

$$L = 01^*0$$



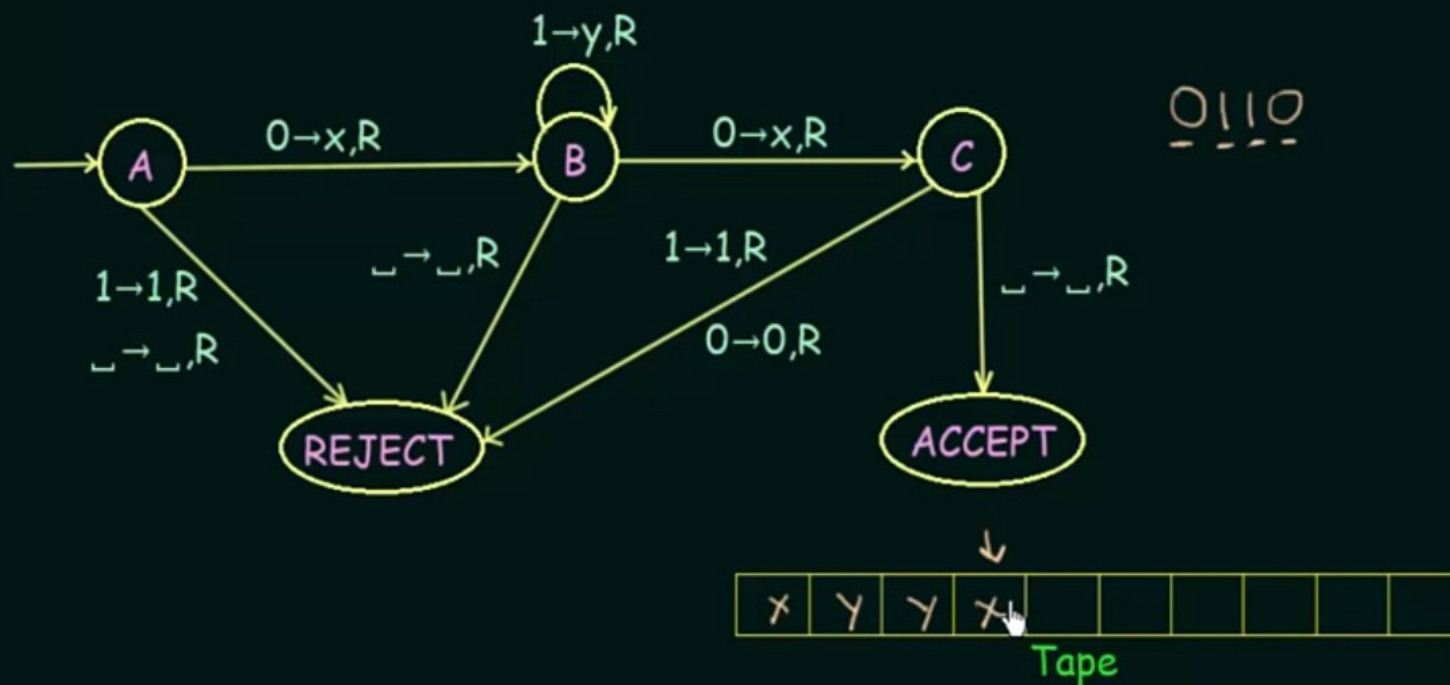
Turing Machine - Example (Part-1)

Design a Turing Machine which recognizes the language
 $L = 01^*0$



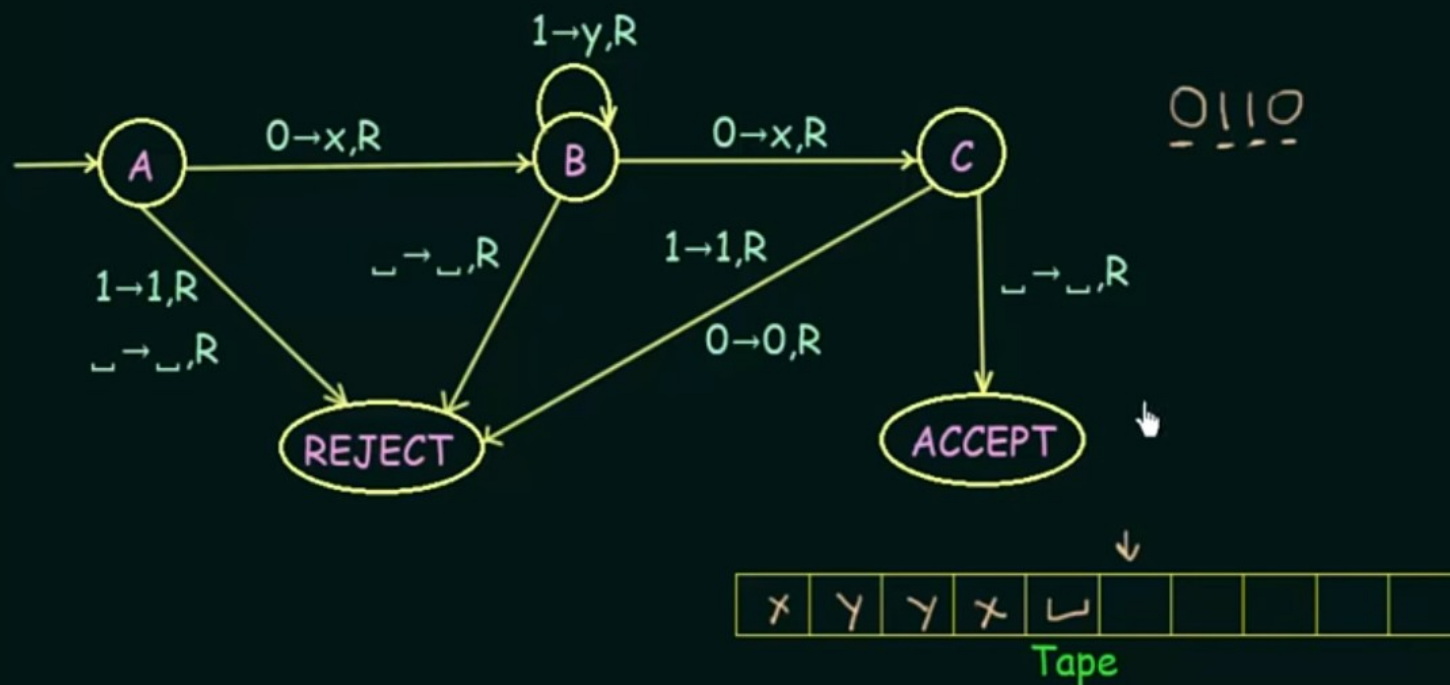
Turing Machine - Example (Part-1)

Design a Turing Machine which recognizes the language
 $L = 01^*0$



Turing Machine - Example (Part-1)

Design a Turing Machine which recognizes the language
 $L = 01^*0$



Turing Machine - Example (Part-1)

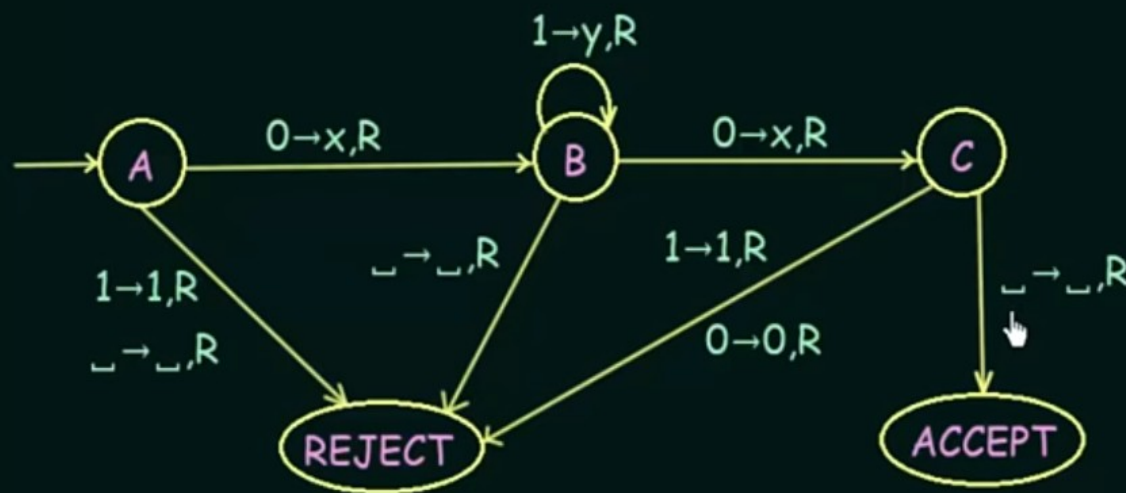
Design a Turing Machine which recognizes the language

$$L = 01^*0$$

$$\Sigma = \{0, 1\}$$

$$b = \sqcup$$

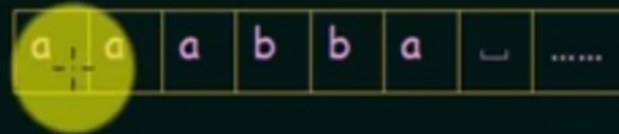
0110 ✓



Turing Machine Programming Techniques (Part-1)

Problem: How can we recognize the left end of the Tape of a Turing Machine ?

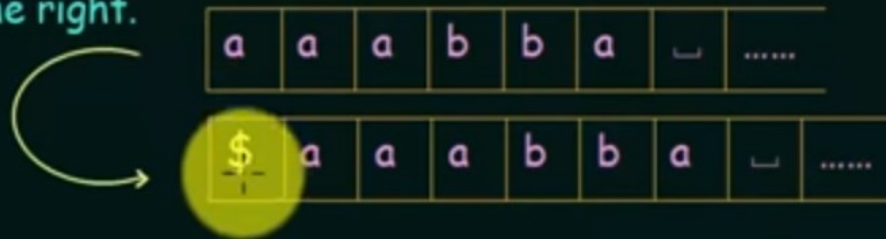
Solution: Put a Special Symbol \$ on the left end of the Tape and shift the input over one cell to the right.



Turing Machine Programming Techniques (Part-1)

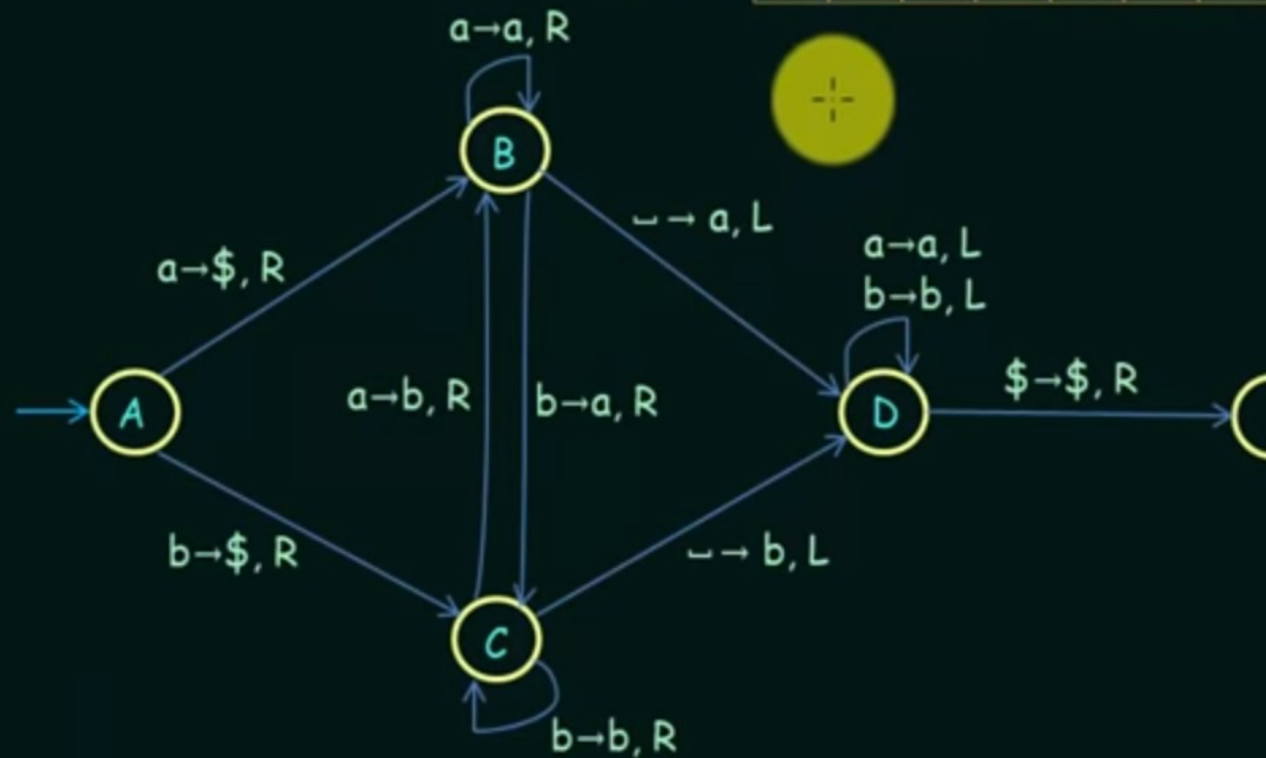
Problem: How can we recognize the left end of the Tape of a Turing Machine ?

Solution: Put a Special Symbol \$ on the left end of the Tape and shift the input over one cell to the right.



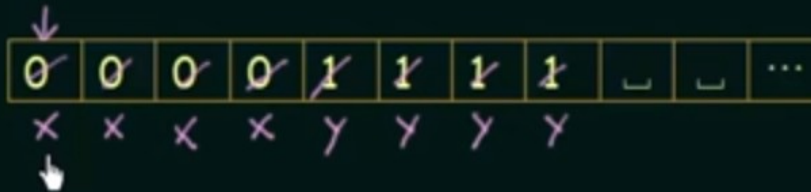
over one cell to the right.

a	a	a	b	b	a	␣		
\$	a	a	a	b	b	a	␣	



Turing Machine - Example (Part-2)

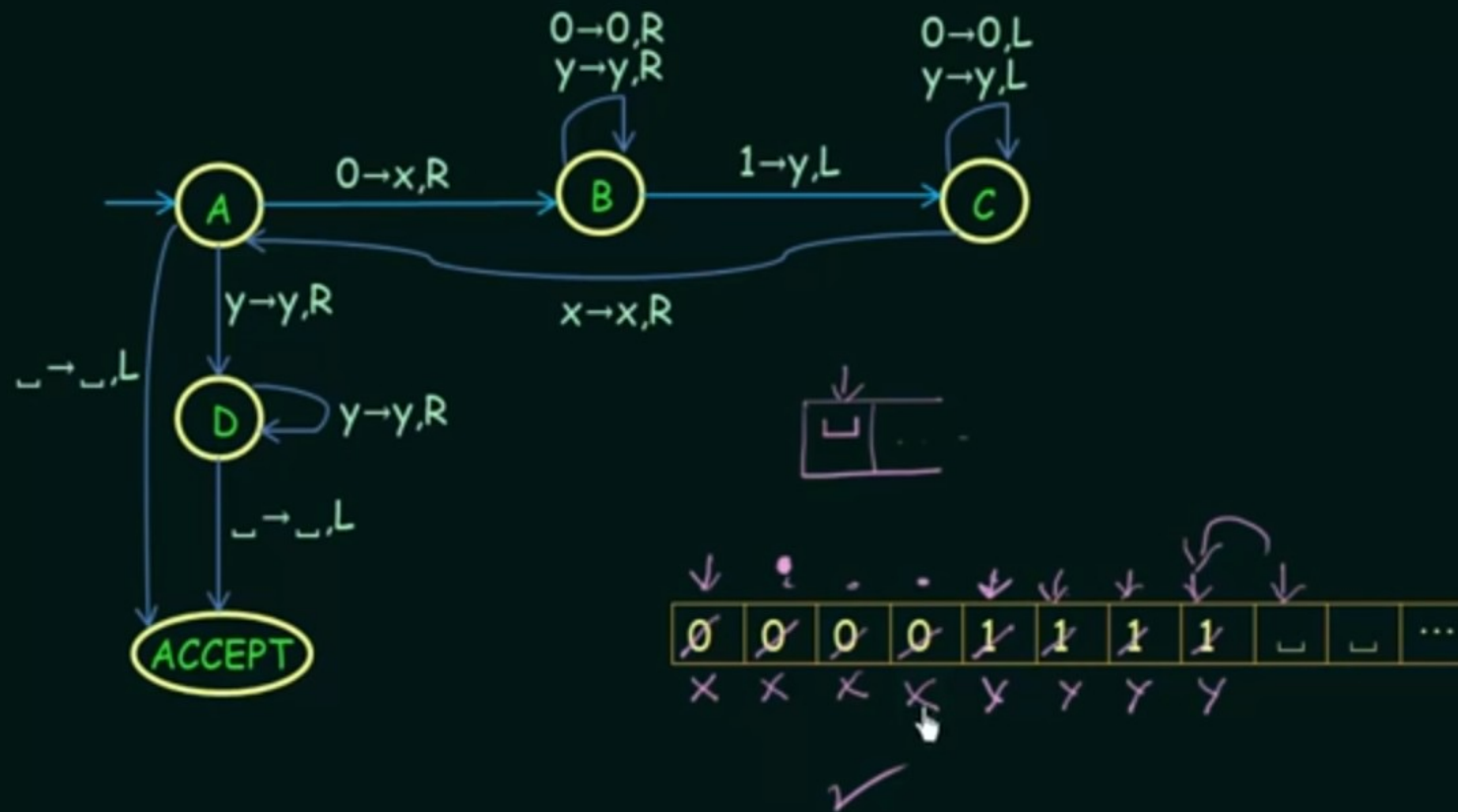
Design a Turing Machine which recognizes the language $L = 0^N 1^N$



Algorithm:

- Change "0" to "x"
- Move RIGHT to First "1"
 If None: **REJECT**
- Change "1" to "y"
- Move LEFT to Leftmost "0"
- Repeat the above steps until no more "0"s
- Make sure no more "1"s remain





Turing Machine - Example (Part-2)

Design a Turing Machine which recognizes the language $L = 0^N 1^N$

0	0	0	0	1	1	1	1	␣	␣	...
---	---	---	---	---	---	---	---	---	---	-----



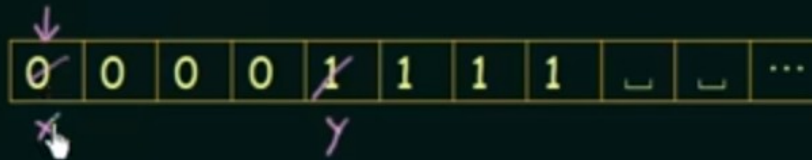
Algorithm:

- Change "0" to "x"
- Move RIGHT to First "1"
If None: **REJECT**
- Change "1" to "y"
- Move LEFT to Leftmost "0"
- Repeat the above steps until no more "0"s
- Make sure no more "1"s remain



Turing Machine - Example (Part-2)

Design a Turing Machine which recognizes the language $L = 0^N 1^N$



Algorithm:

- Change "0" to "x"
- Move RIGHT to First "1"
If None: **REJECT**
- Change "1" to "y"
- Move LEFT to Leftmost "0"
- Repeat the above steps until no more "0"s
- Make sure no more "1"s remain



Turing Machine - Example (Part-2)

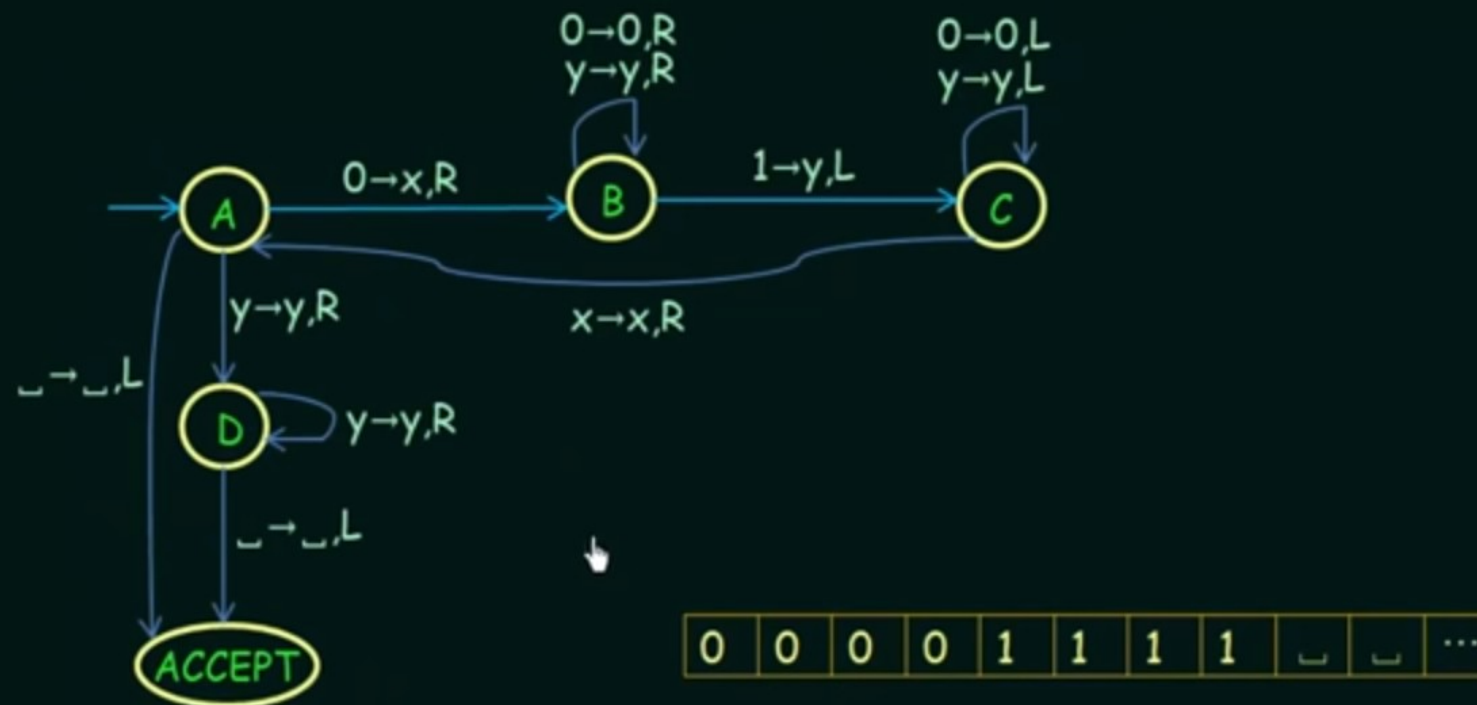
Design a Turing Machine which recognizes the language $L = 0^N 1^N$

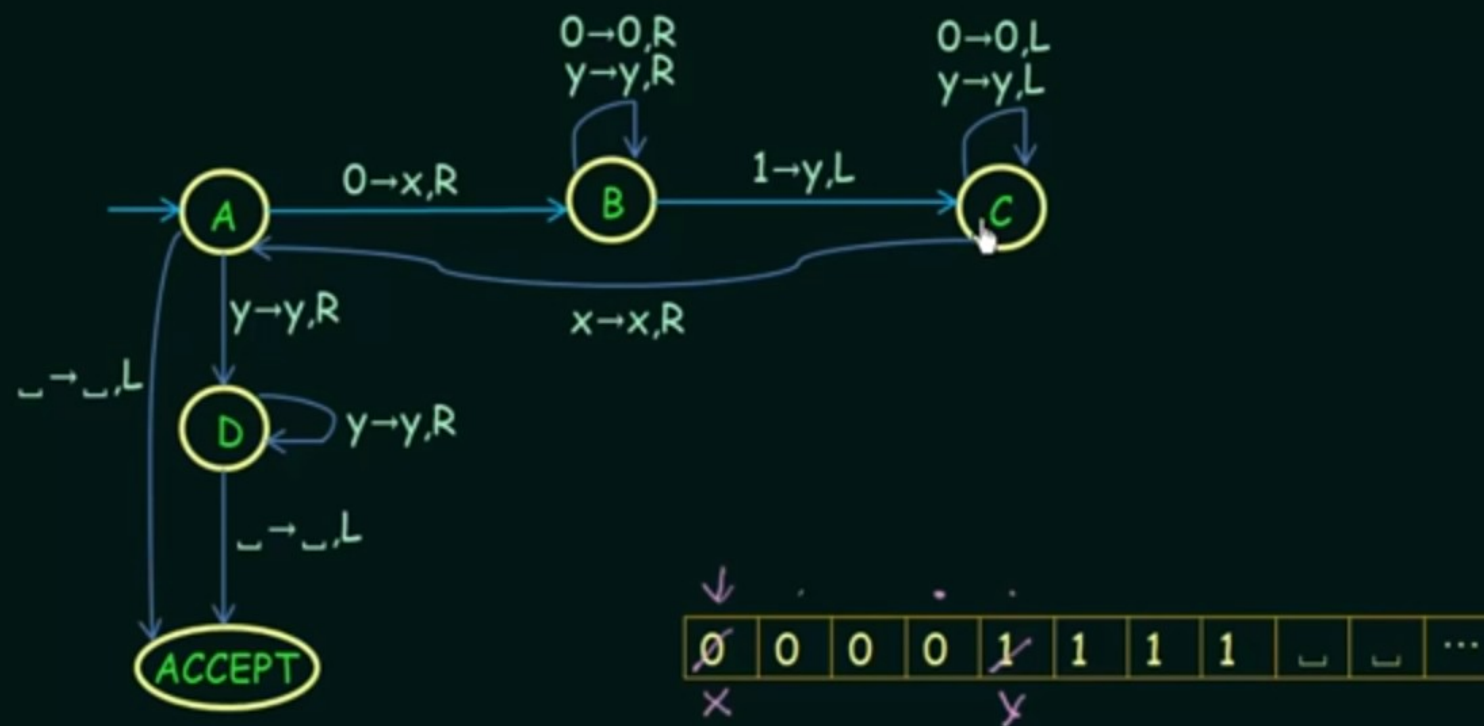


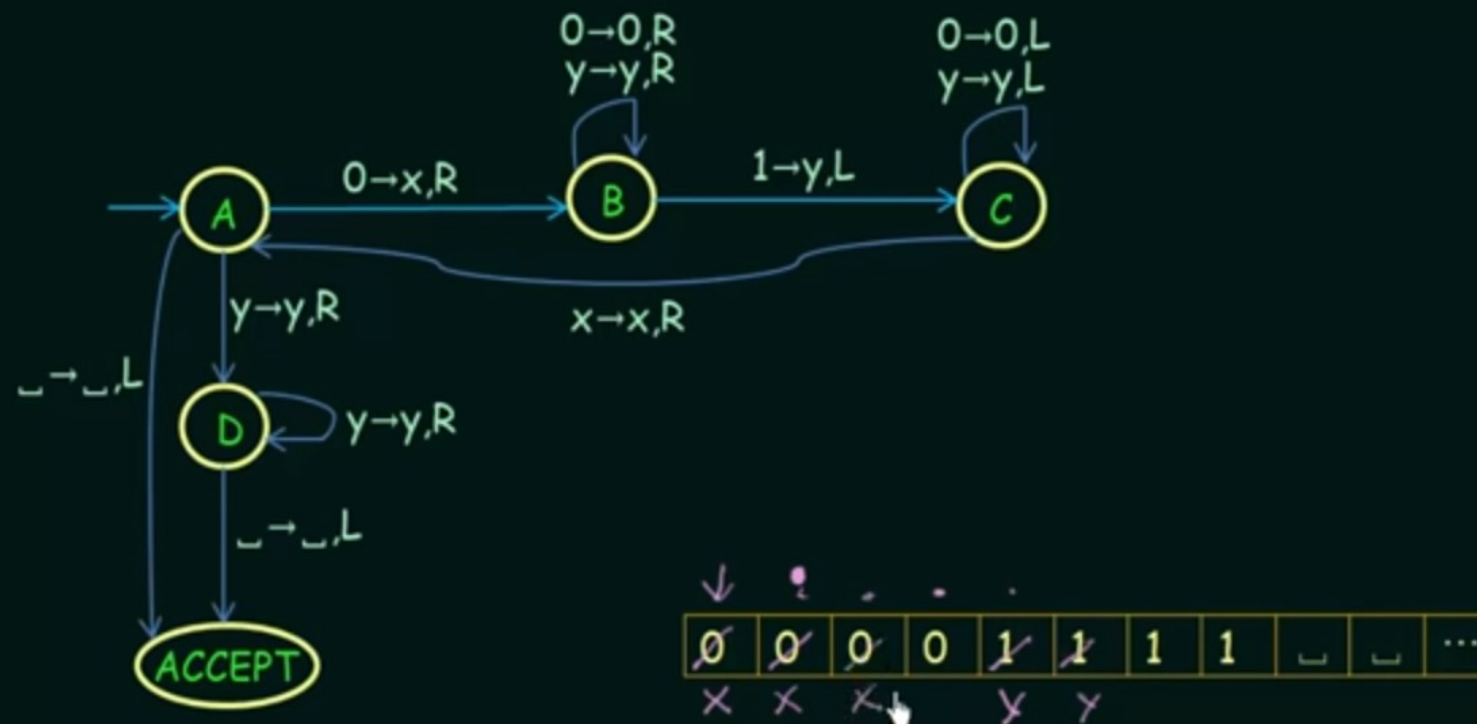
Algorithm:

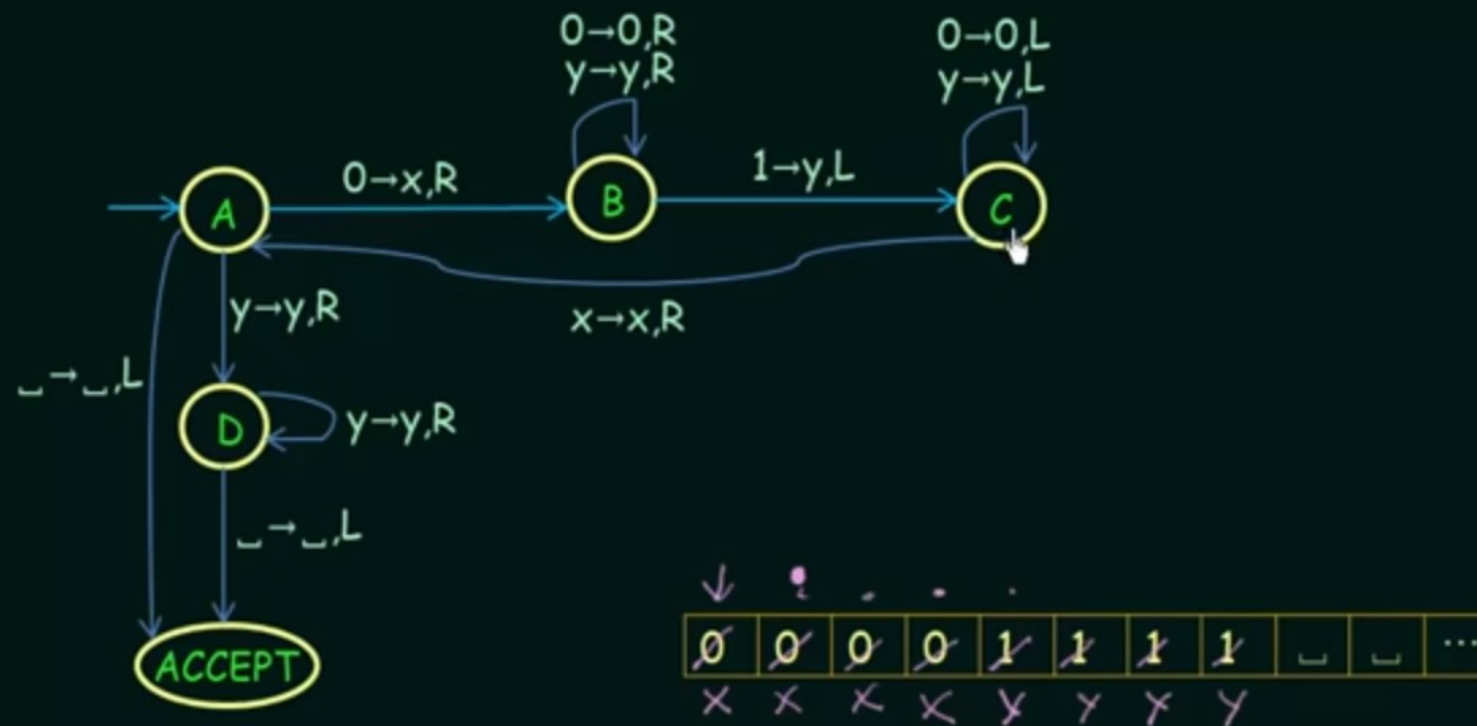
- Change "0" to "x"
- Move RIGHT to First "1"
If None: **REJECT**
- Change "1" to "y"
- Move LEFT to Leftmost "0"
- Repeat the above steps until no more "0"s
- Make sure no more "1"s remain

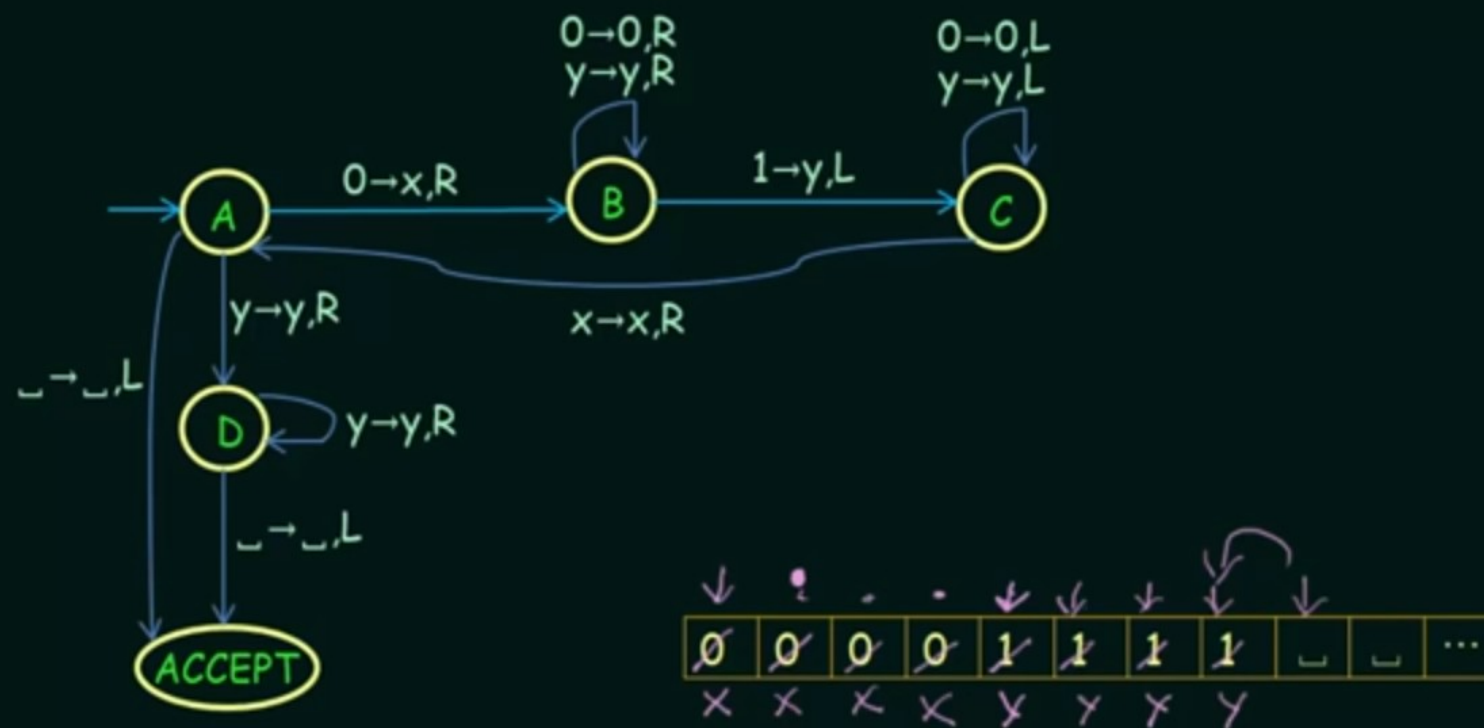












Turing Machine as Problem Solvers

Any arbitrary Problem can be expressed as a language

-Any instance of the problem is encoded into a string

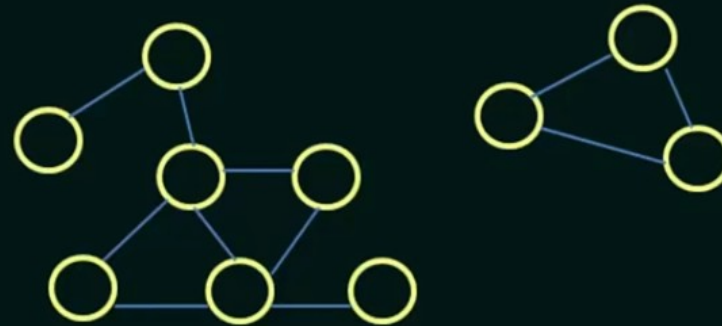
The string is in the language $\Rightarrow$ The answer is YES

The string is not in the language $\Rightarrow$ The answer is NO

Example: Is this undirected graph connected?



Example: Is this undirected graph connected?



We must encode the problem into a language.

$A = \{ \langle G \rangle \mid G \text{ is a connected graph} \}$

We would like to find a TM to decide this language:

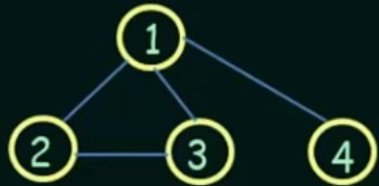
ACCEPT = "YES", This is a connected graph

REJECT = "NO", This is not a connected graph / or this is not a valid
Representation of a graph.

LOOP = This problem is decidable. Our TM will always halt



Representation of Graph:



$\langle G \rangle = (1, 2, 3, 4) \quad ((1, 2), (2, 3), (1, 3), (1, 4))$

 List of nodes Edges

$\Sigma = \{ (,), , 1, 2, 3, 4, \dots, 0 \}$

(	1	,	2	,	3	,	4	,		)	)	_	
---	---	---	---	---	---	---	---	---	-------	---	---	---	-------



High Level Algorithm:

Select a Node and Mark it

REPEAT

```
┌ For each node N
│   If N is unmarked and there is an edge from N to an already marked
│   node
│   Then
│     Mark Node N
└ End
```

Until no more nodes can be marked

```
┌ For each Node N
│   If N is unmarked
│   Then REJECT
└ End
```

ACCEPT



High Level Algorithm:

Select a Node and Mark it

REPEAT

```
┌ For each node N
│   If N is unmarked and there is an edge from N to an already marked
│   node
│   Then
│     Mark Node N
└ End
```

Until no more nodes can be marked

```
┌ For each Node N
│   If N is unmarked
│   Then REJECT
└ End
```

ACCEPT



Implementation Level Algorithm:

- Check that input describes a valid graph
- Check Node List
 - Scan "(" followed by digits ...
 - Check that all nodes are different i.e. no repeats
 - Check edge lists ...
 - etc.
- Mark First Node
 - Place a dot under the first node in the node list
 - Scan the node list to find a node that is not marked
 - etc.



Implementation Level Algorithm:

- Check that input describes a valid graph
- Check Node List
 - Scan "(" followed by digits ...
 - Check that all nodes are different i.e. no repeats
 - Check edge lists ...
 - etc.
- Mark First Node
 - Place a dot under the first node in the node list
 - Scan the node list to find a node that is not marked
 - etc.



Decidability and Undecidability

Recursive Language:

- A language 'L' is said to be recursive if there exists a Turing machine which will accept all the strings in 'L' and reject all the strings not in 'L'.
- The Turing machine will halt every time and give an answer (accepted or rejected) for each and every string input.

Recursively Enumerable Language:

- A language 'L' is said to be a recursively enumerable language if there exists a Turing machine which will accept (and therefore halt) for all the input strings which are in 'L'.
- But may or may not halt for all input strings which are not in 'L'.



Decidable Language:

A language 'L' is decidable if it is a recursive language. All decidable languages are recursive languages and vice-versa.

Partially Decidable Language:

A language 'L' is partially decidable if 'L' is a recursively enumerable language.

Undecidable Language:

- A language is undecidable if it is not decidable.
- An undecidable language may sometimes be partially decidable but not decidable.
- If a language is not even partially decidable, then there exists no Turing machine for that language

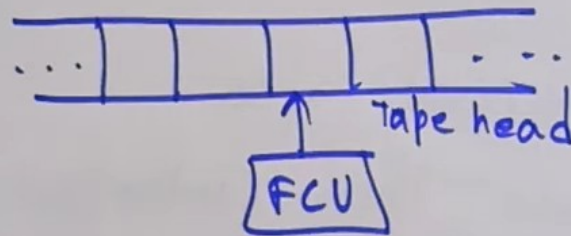


Recursive Language	TM will always Halt
Recursively Enumerable Language	TM will halt sometimes & may not halt sometimes
Decidable Language	Recursive Language
Partially Decidable Language	Recursively Enumerable Language
UNDECIDABLE	No TM for that language



Variants of Turing Machine

⇒ Till now we have seen normal T.M with tape infinite size, Tape head & one finite control unit

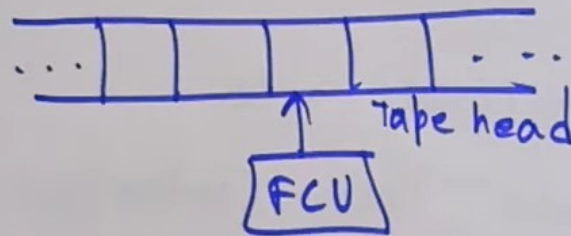


⇒ Different Variations of Turing Machine

- Multitape Turing Machines
- Non Deterministic Turing Machines

Variants of Turing Machine

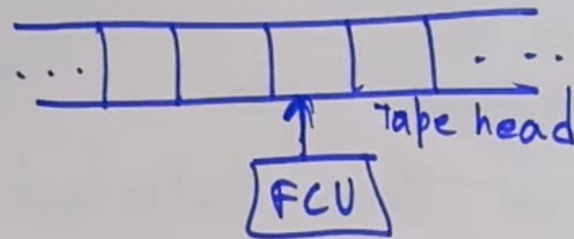
⇒ Till now we have seen normal T.M with tape infinite size, Tape head & one finite control unit



⇒ Different Variations of Turing Machine

- Multitape Turing Machines
- Non Deterministic Turing Machines

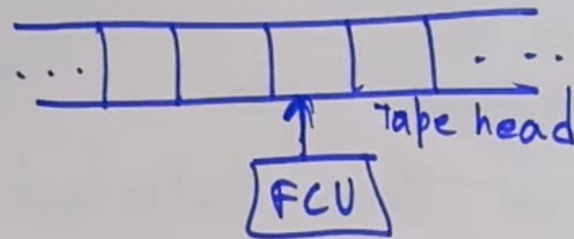
⇒ Till now we have seen normal T.M with tape infinite size, Tape head & one finite control unit



⇒ Different Variations of Turing Machine

- Multitape Turing Machines
- Non Deterministic Turing Machines
- Multihead Turing Machines

⇒ Till now we have seen normal T.M with tape infinite size, Tape head & one finite control unit



⇒ Different Variations of Turing Machine

- Multitape Turing Machines
- Non Deterministic Turing Machines
- Multihead Turing Machines

- off line Turing machines
- Multidimensional Turing machines

① Multitape TM :

- A T.M with several tapes
- Every Tapes have their controlled own R/W Head
- For N -tape T.M $M = \{Q, \Sigma, X, \delta, q_0, B, F\}$

We define $\delta = Q \times X^N \rightarrow Q \times X^N \times \{L, R\}^N$

- off line Turing machines
- Multidimensional Turing machines

① Multitape TM :

- A T.M with several tapes
- Every Tapes have their controlled own R/W Head
- For N -tape T.M $M = \{Q, \Sigma, X, \delta, q_0, B, F\}$

We define $\delta = Q \times X^N \rightarrow Q \times X^N \times \{L, R\}^N$

→ off line Turing Machines

→ Multidimensional Turing Machines

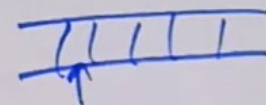
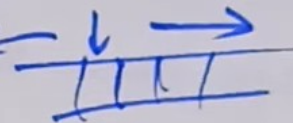
① Multitape TM :

→ A T.M with several tapes

→ Every Tapes have their controlled own R/W Head

→ For N-tape T.M $\Gamma = \{0, 1, x, y, z, \dots, B, F\}$

We define $\delta: Q \times X^N \rightarrow Q \times X^N \times \{L, R\}^N$



→ off line Turing Machines

→ Multidimensional Turing Machines

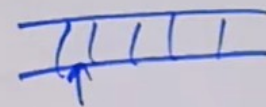
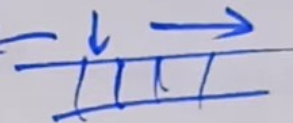
① Multitape TM :

→ A T.M with several tapes

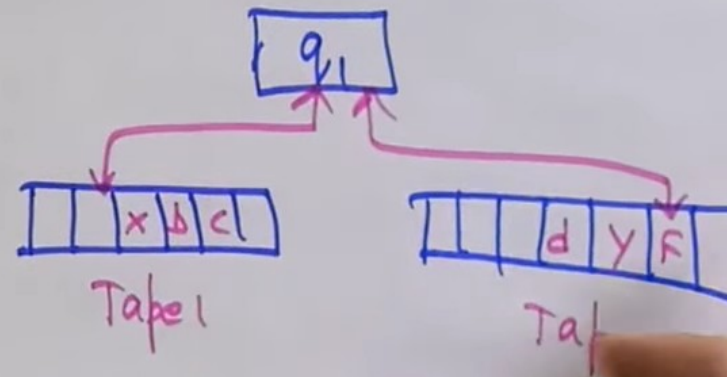
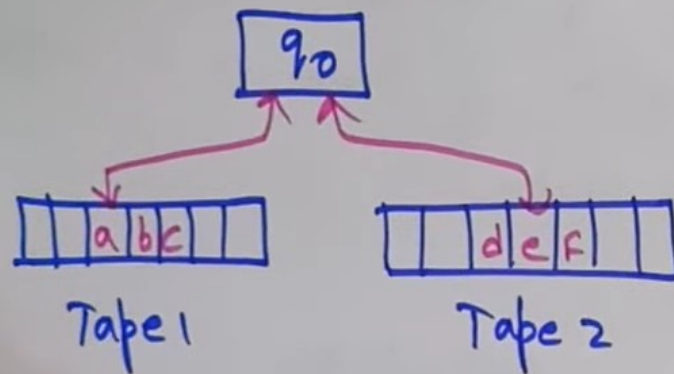
→ Every Tapes have their controlled own R/W Head

→ For N-tape T.M $\Gamma = \{0, 1, x, y, z, \dots, B, F\}$

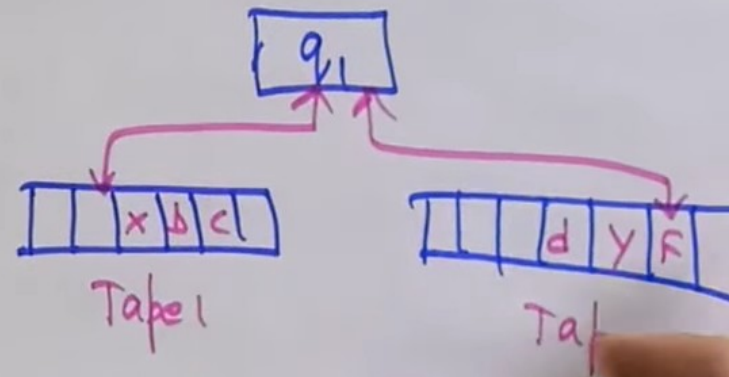
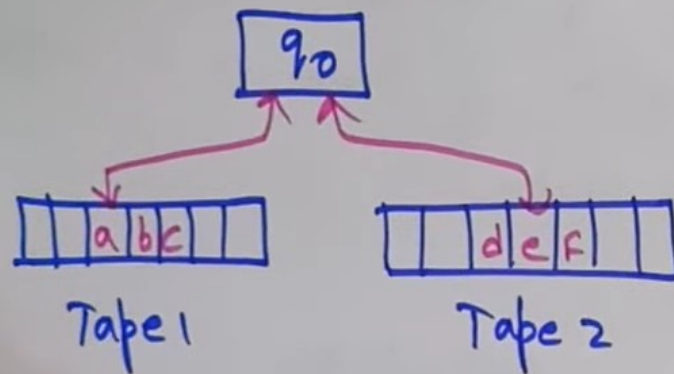
We define $\delta: Q \times X^N \rightarrow Q \times X^N \times \{L, R\}^N$



For eg: if $n=2$, with current configuration
 $\delta(q_0, a, e) = (q_1, x, y, L, R)$



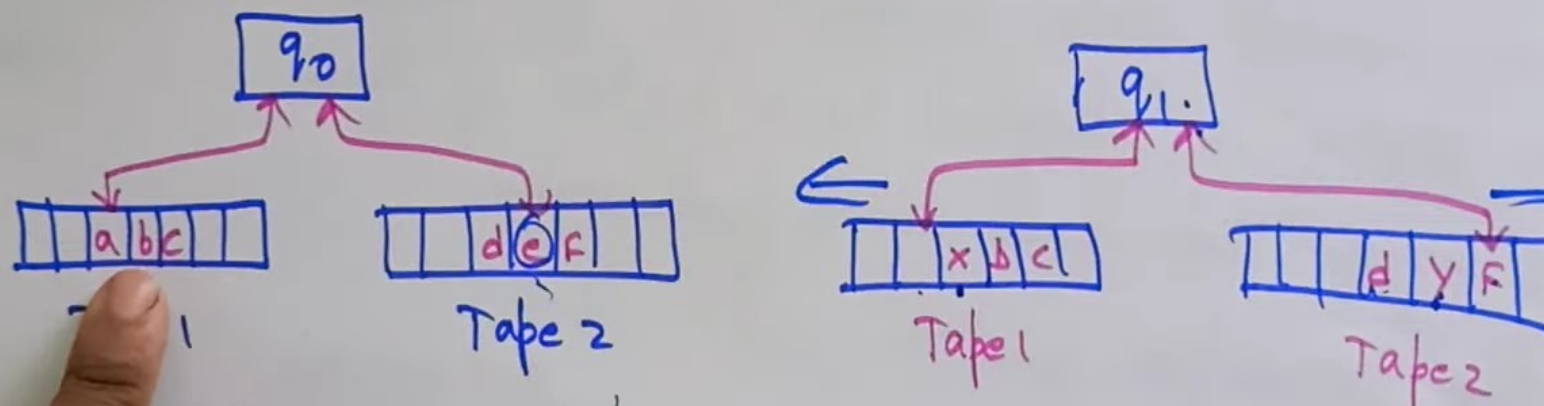
For eg: if $n=2$, with current configuration
 $\delta(q_0, a, e) = (q_1, x, y, L, R)$



For eg: if $n=2$, with current configuration

$$\delta(q_0, a, e) = (q_1, x, y, L, R)$$

$N=2$

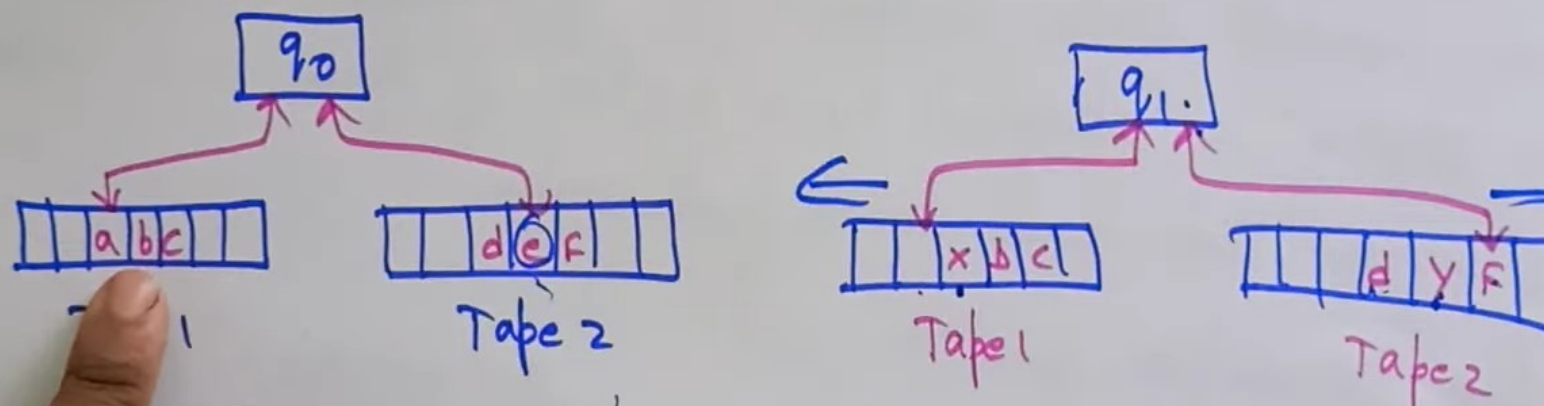


$$\delta: Q \times \Sigma^N \rightarrow Q \times \Sigma^N \times \{L, R\}^N$$

For eg: if $n=2$, with current configuration

$$\delta(q_0, a, e) = (q_1, x, y, L, R)$$

$N=2$



$$\delta: Q \times \Sigma^N \rightarrow Q \times \Sigma^N \times \{L, R\}^N$$

② Non Deterministic Turing Machine ↓

→ It is similar to DTM except that for any i/p s and current state it has a no: of choices

→ A string is accepted by a NDTM if there is a seq of moves that leads to a final state.

→ The transition function

$$\delta: Q \times X \rightarrow 2^{Q \times X \times \{L, R\}}$$

② Non Deterministic Turing Machine $\downarrow$

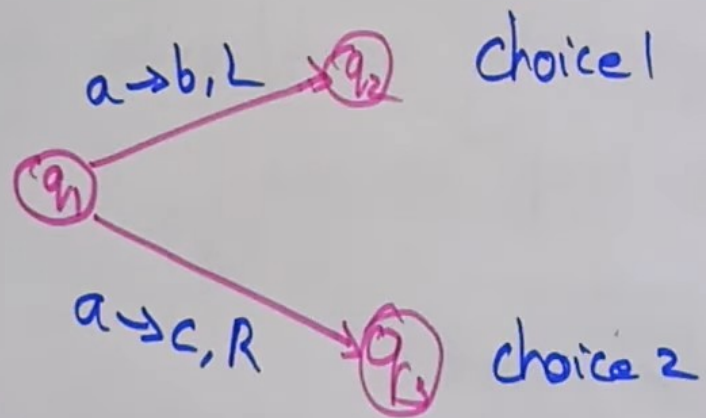
→ It is similar to DTM except that for any i/p ϵ and current state it has a no: of choices

→ A string is accepted by a NDTM if there is a seq of moves that leads to a final state.

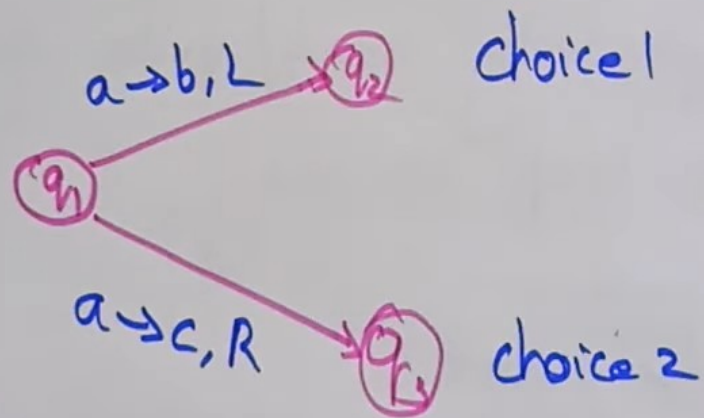
→ The transition function

$$\delta: Q \times X \rightarrow 2^{Q \times X \times \{L, R\}}$$

→ A NDTM is allowed to have more than one transition on a given tape symbol.

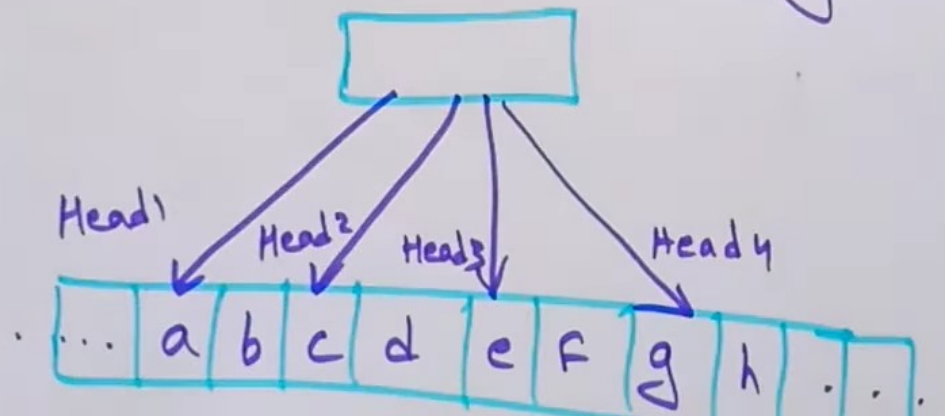


→ A NDTM is allowed to have more than one transition on a given tape symbol.



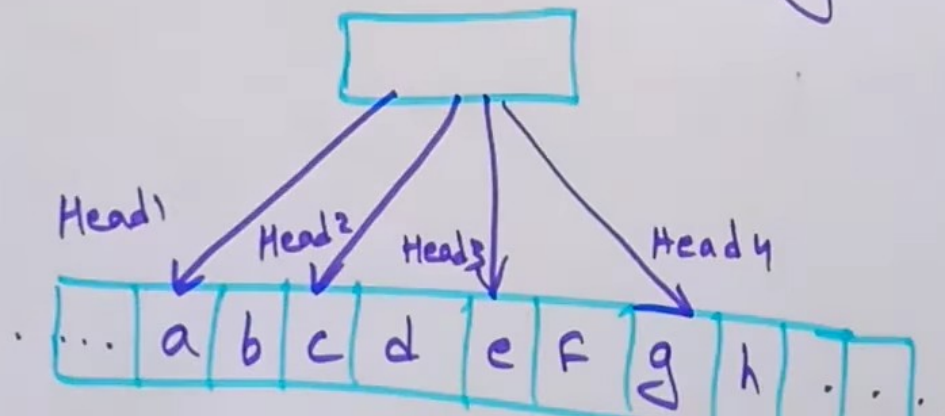
③ Multihead TM

- It has no. of heads instead of one.
- Each head independently read/write symbols and move left/right or keep stationary



③ Multihead TM

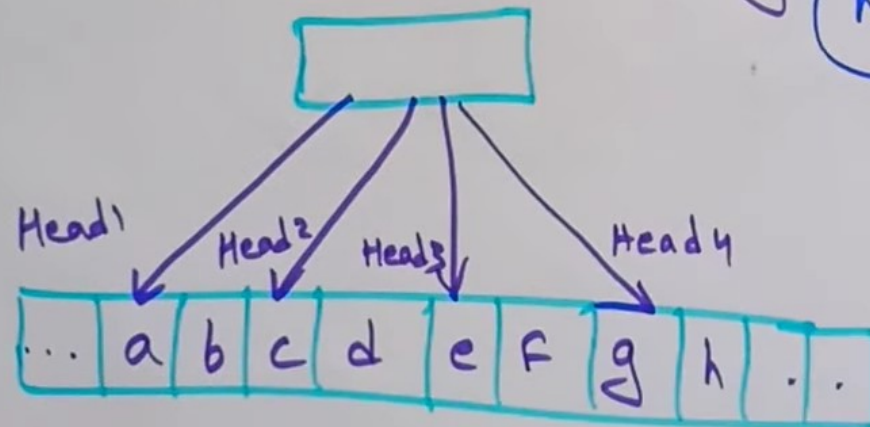
- It has no. of heads instead of one.
- Each head independently read/write symbols and move left/right or keep stationary



③ Multihead TM :-

→ It has no. of heads instead of one.

→ Each head independently read/write symbols and move left/right or keep stationary Independently

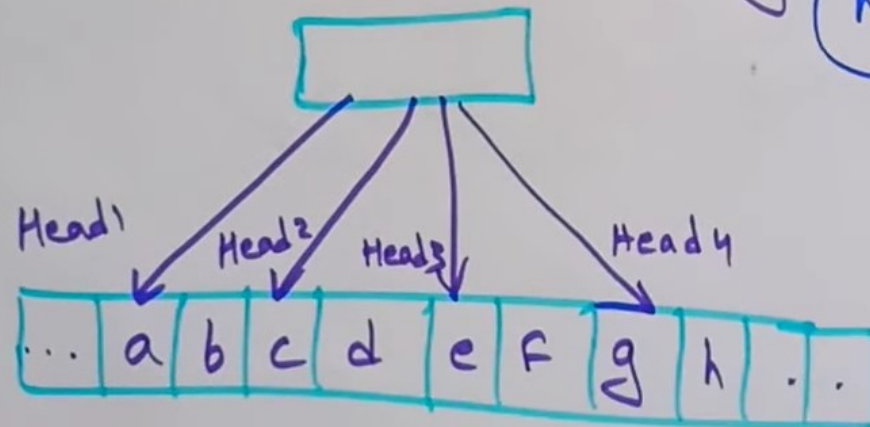


head 1 — R/w
2 — R/w
3
4

③ Multihead TM :-

→ It has no. of heads instead of one.

→ Each head independently read/write symbols and move left/right or keep stationary Independently



head 1 — R/w
2 — R/w
3
4

Multi track TM

1st track	...	1	B	B	B	B	B	B	B	...
2nd	...	B	B	1	B	B	B	B	B	...
3rd	...	B	B	B	B	1	B	B	B	...
4th	...	B	B	B	B	B	B	1	B	...
5th	...	b	c	d	e	f	g	h	...	...

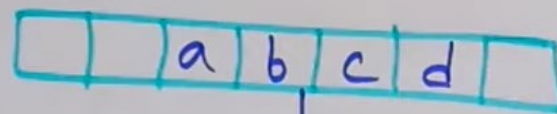
Multi track TM

1st track	...	1	B	B	B	B	B	B	B	...
2nd	...	B	B	1	B	B	B	B	B	...
3rd	...	B	B	B	B	1	B	B	B	...
4th	...	B	B	B	B	B	B	1	B	...
5th	...	b	c	d	e	f	g	h	...	...

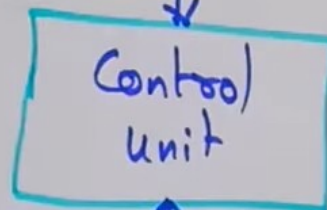
④ Off-line Turing Machine

→ An off line TM has two tapes

- ① One tape is read-only and contains the i/p
- ② The other is read-write and is initially blank.



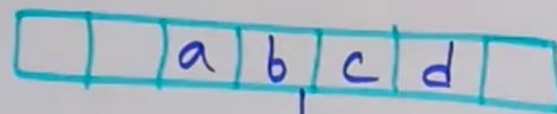
Read-only i/p
file's tape



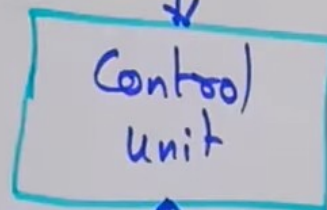
④ Off-line Turing Machine

→ An off line TM has two tapes

- ① One tape is read-only and contains the i/p
- ② The other is read-write and is initially blank.



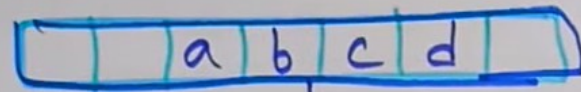
Read-only i/p
file's tape



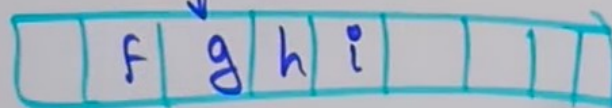
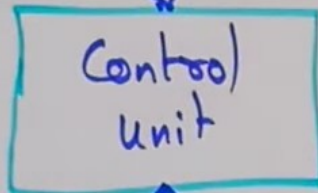
→ An off line TM has two tapes

- ① One tape is read-only and contains the i/p
- ② The other is read-write and is initially blank.

Read



Read-only i/p
file's tape

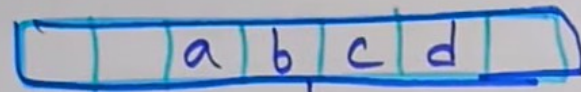


W/R tape

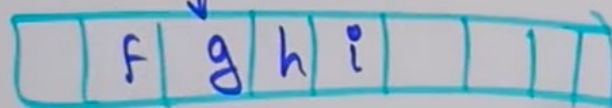
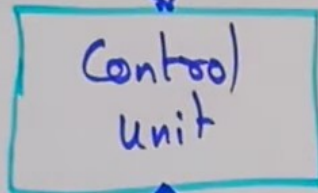
→ An off line TM has two tapes

- ① One tape is read-only and contains the i/p
- ② The other is read-write and is initially blank.

Read



Read-only i/p
file's tape



W/R tape

⑤ Multidimensional Turing Machine

→ Multidimensional TM has a multidimensional tape.

fr eg 1. a two-dimensional T.M would read and write on an infinite tape divided into squares, like a checkboard.

→ In a two dimensional TM transaction function defined as:

$$\delta: Q \times T \rightarrow Q \times T \times \{L, R, U, D\}$$

⑤ Multidimensional Turing Machine

→ Multidimensional TM has a multidimensional tape.

fr eg 1. a two-dimensional T.M would read and write on an infinite tape divided into squares, like a checkboard.

→ In a two dimensional TM transaction function defined as:

$$\delta: Q \times T \rightarrow Q \times T \times \{L, R, U, D\}$$

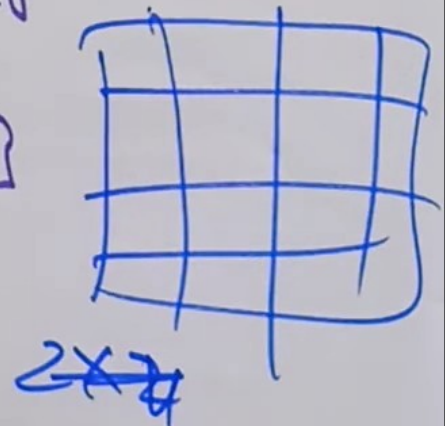
⑤ Multidimensional Turing Machine

→ Multidimensional TM has a multidimensional tape.

fr eg 1. a two-dimensional T.M would read and write on an infinite tape divided into squares, like a checkboard.

→ In a two dimensional TM transaction function defined as:

$$\textcircled{8} \quad Q \times T \rightarrow Q \times T \times \{L, R, U, D\}$$



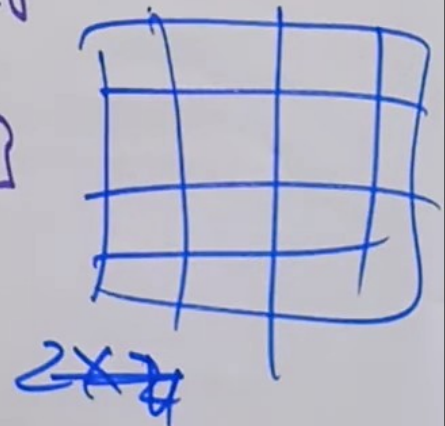
⑤ Multidimensional Turing Machine

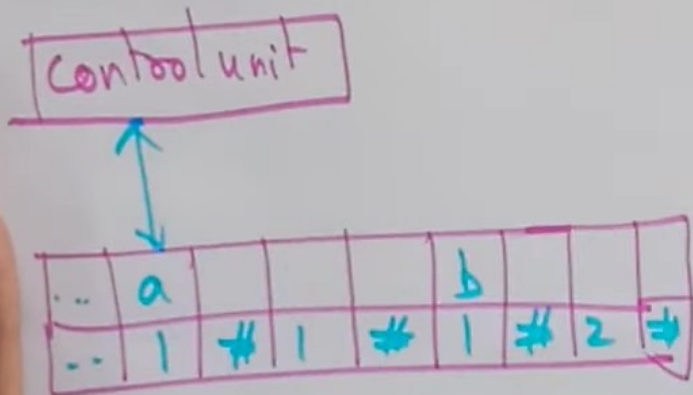
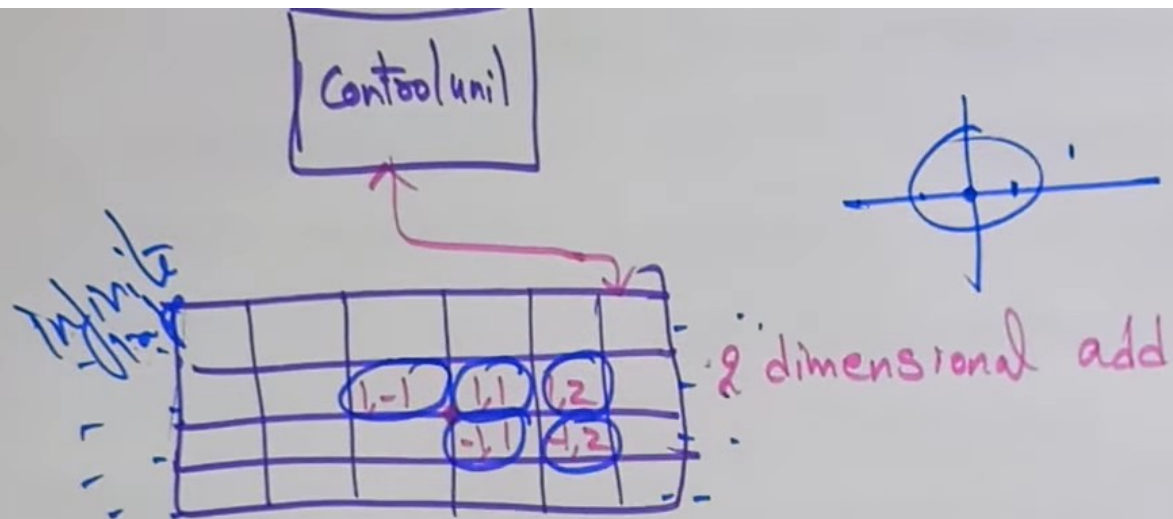
→ Multidimensional TM has a multidimensional tape.

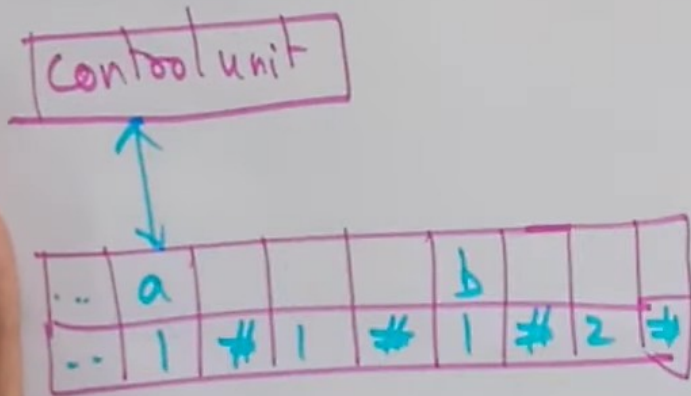
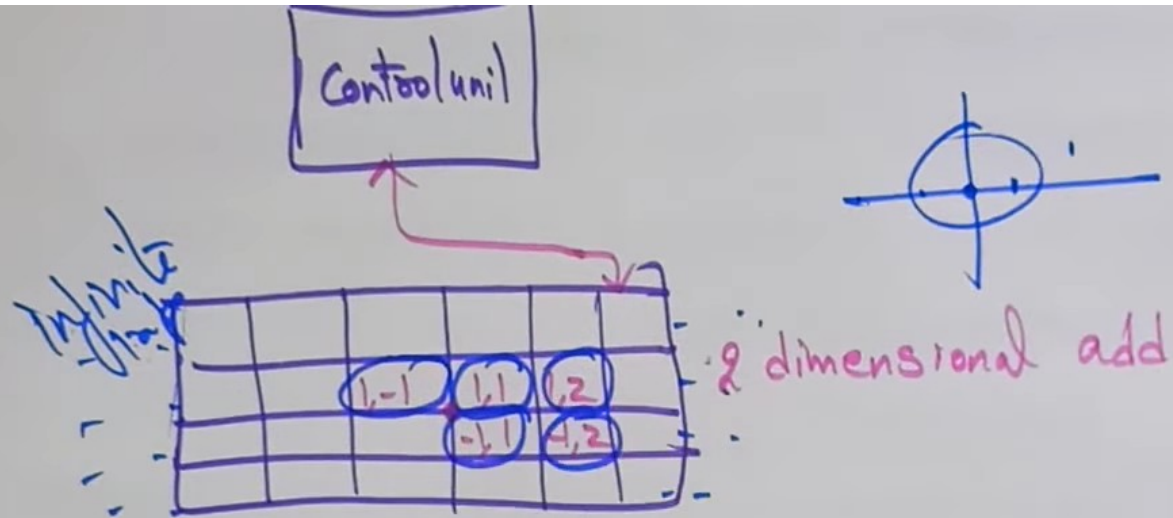
fr eg 1. a two-dimensional T.M would read and write on an infinite tape divided into squares, like a checkboard.

→ In a two dimensional TM transaction function defined as:

$$\textcircled{8} \quad Q \times T \rightarrow Q \times T \times \{L, R, U, D\}$$







Recursive & Recursively Enumerable language

→ The T.M may

- * Halt & accept the input
- * Halt & reject the input, $\emptyset$
- * Never halt / loop

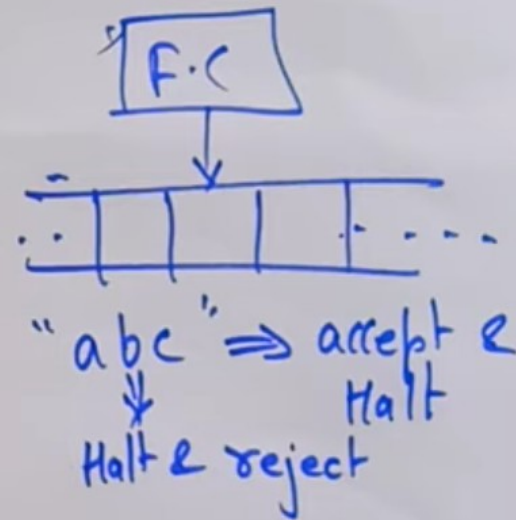
Recursively Enumerable lang :-

There is a T.M for a language which accept every string otherwise not.

Recursive & Recursively Enumerable languages

→ The T.M may

- * Halt & accept the input
- * Halt & reject the input, $\emptyset$
- * Never halt / loop



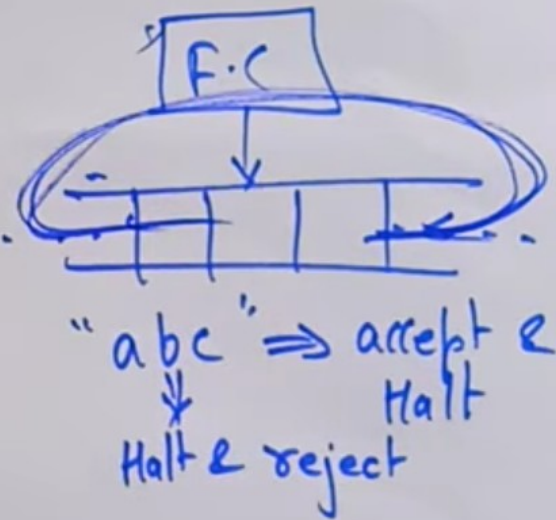
Recursively Enumerable lang :-

There is a T.M for a language which accepts every string otherwise not.

Recursive & Recursively Enumerable languages

→ The T.M may

- * Halt & accept the input
- * Halt & reject the input, $\emptyset$
- * Never halt / loop



Recursively Enumerable lang :-

There is a T.M for a language which accepts every string otherwise not.

Recursive language L

There is a T.M for a language which halt
on every string

Recursive language $\hookleftarrow$

There is a T.M for a language which halt
on every string

Recursive lang
Halt on every
string

recursively enumerable lang
Accept every string or
not

Multitape Turing Machine

Theorem: Every Multitape Turing Machine has an equivalent Single Tape Turing Machine 



Multitape Turing Machine

Theorem: Every Multitape Turing Machine has an equivalent Single Tape Turing Machine

Proof

Given a Multitape Turing Machine show how to build a single tape Turing Machine

- Need to store all tapes on a single tape
Show data representation



Multitape Turing Machine

Theorem: Every Multitape Turing Machine has an equivalent Single Tape Turing Machine

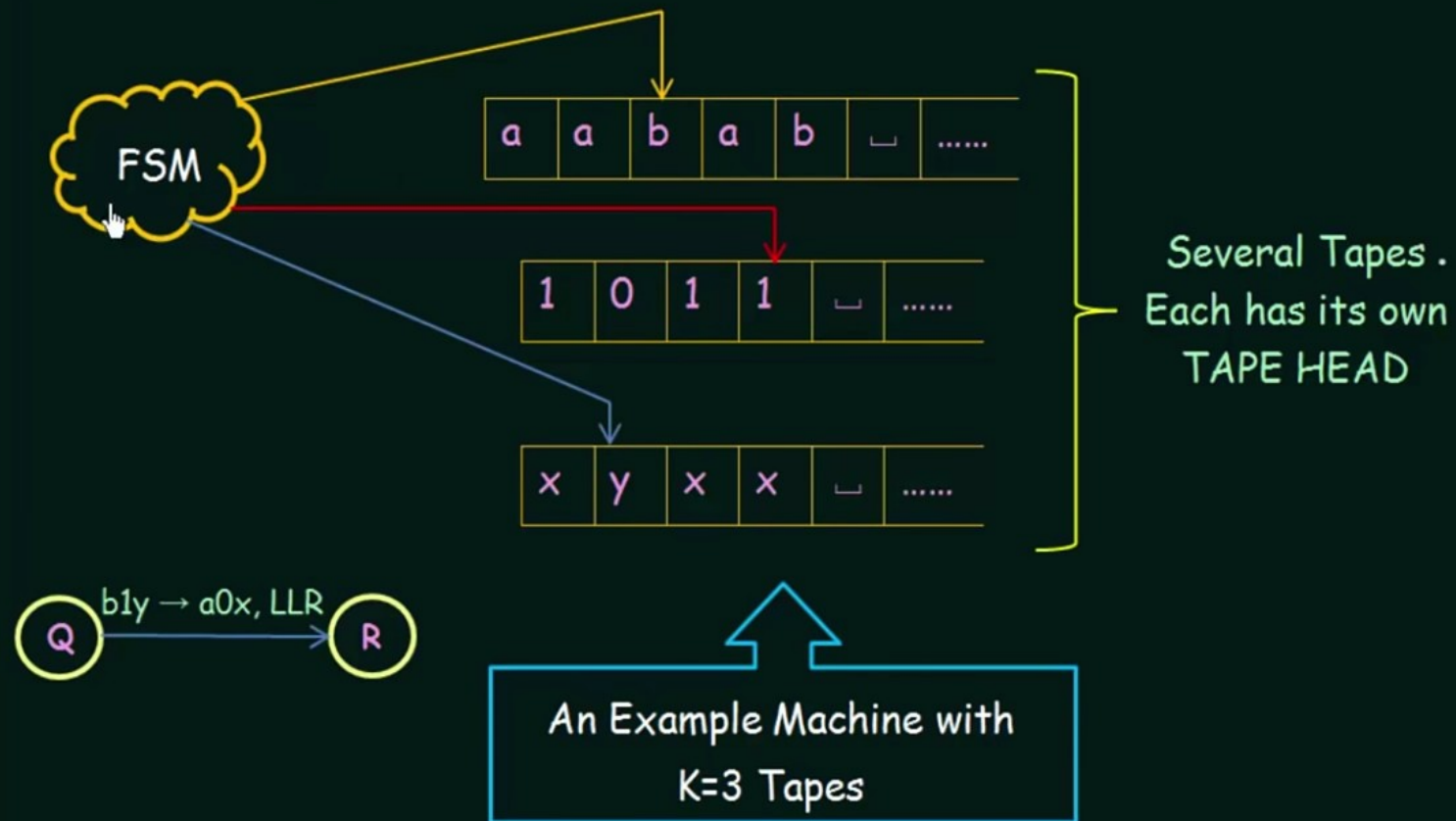
Proof

Given a Multitape Turing Machine show how to build a single tape Turing Machine

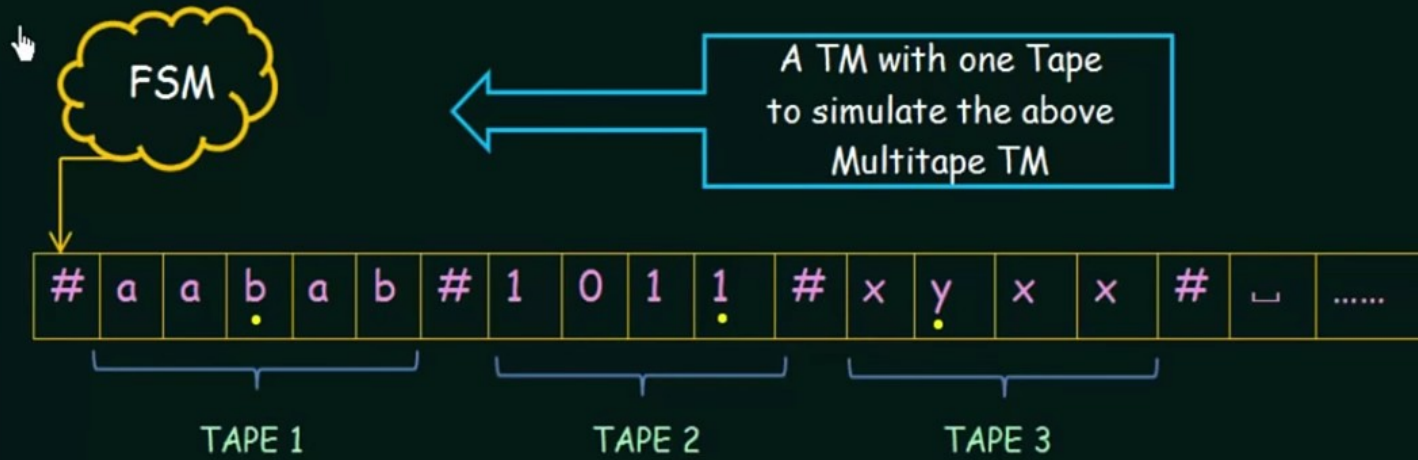
- Need to store all tapes on a single tape
Show data representation
- Each tape has a tape head
Show how to store that info
- Need to transform a move in the Multitape TM into one or moves in the Single Tape TM



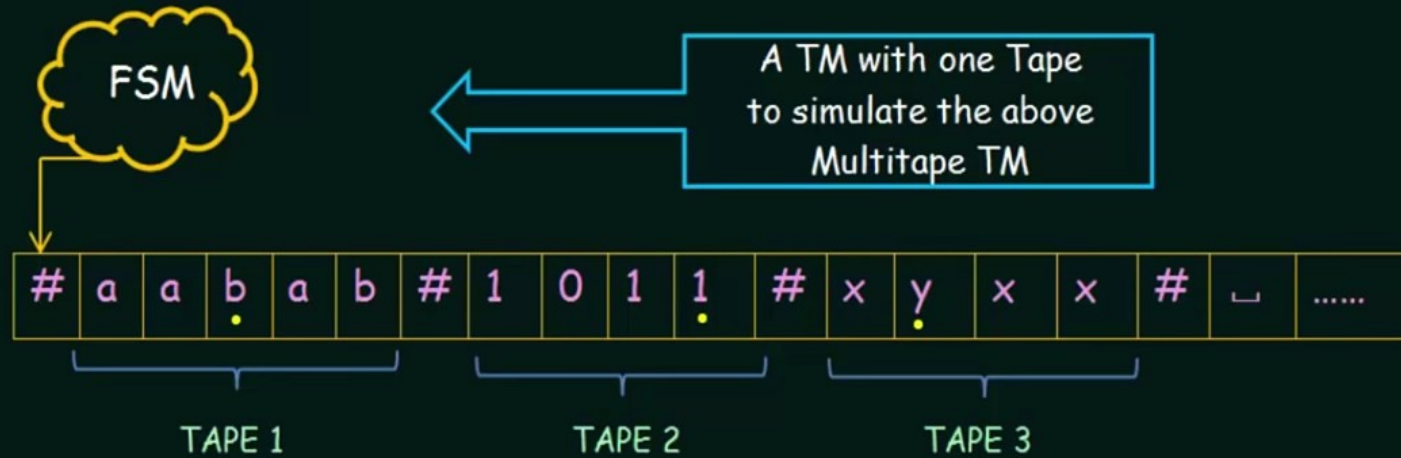
Multitape Turing Machine



Single Tape Turing Machine



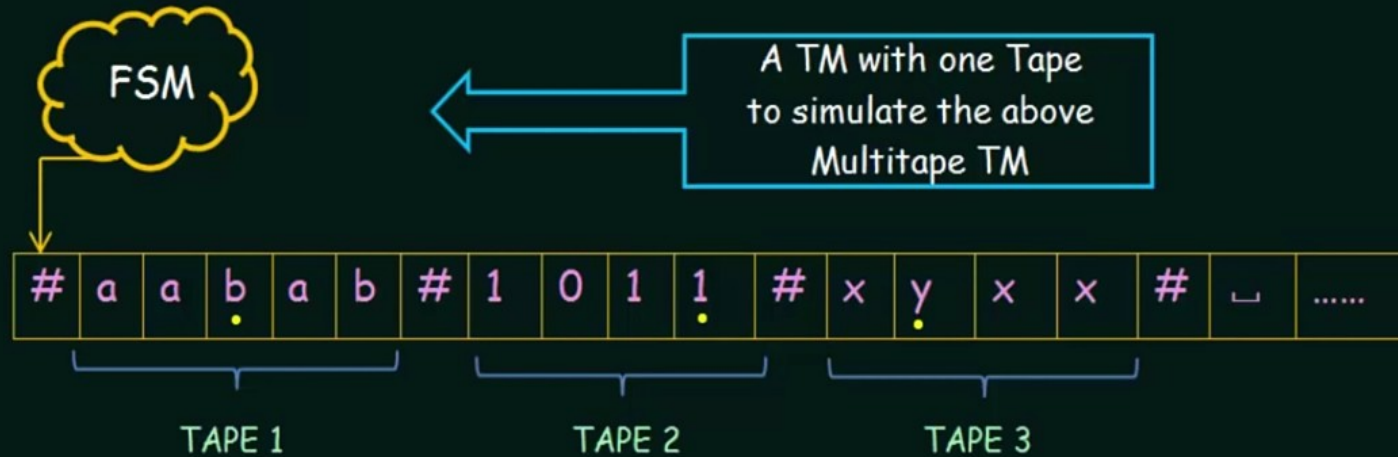
Single Tape Turing Machine



- Add "dots" to show where Head "K" is
- To simulate a transition from state Q , we must scan our Tape to see which symbols are under the K Tape Heads



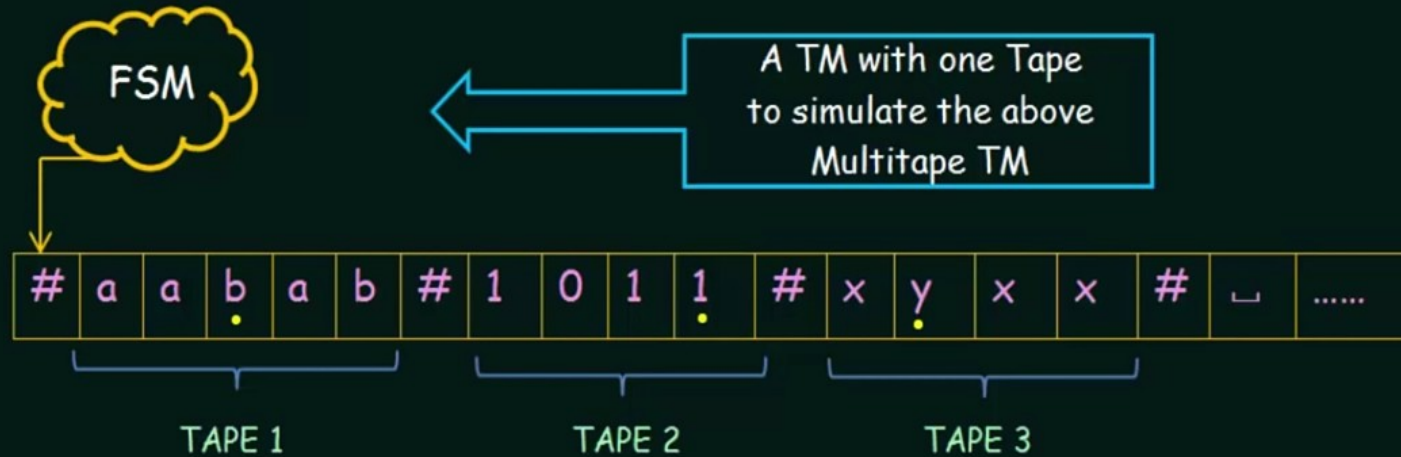
Single Tape Turing Machine



- Add "dots" to show where Head "K" is
- To simulate a transition from state Q , we must scan our Tape to see which symbols are under the K Tape Heads
- Once we determine this and are ready to **MAKE** the transition, we must scan across the tape again to update the cells and move the dots



Single Tape Turing Machine



- Add "dots" to show where Head "K" is
- To simulate a transition from state Q , we must scan our Tape to see which symbols are under the K Tape Heads
- Once we determine this and are ready to **MAKE** the transition, we must scan across the tape again to update the cells and move the dots
- Whenever one head moves off the right end, we must shift our tape so we can insert a □



Turing Machine Programming Techniques (Part-3)

COMPARING TWO STRINGS

A Turing Machine to decide $\{ w \# w \mid w \in \{a,b,c\}^* \}$

Solution:

- Use a new symbol such as 'x'
- Replace each symbol into an x after it has been examined



Solution:

- Use a new symbol such as 'x'
- Replace each symbol into an x after it has been examined

a b b a c # a b b a c



x b b a c # x b b a c



x x b a c # x x b a c



x x x a c # x x x a c



x x x x c # x x x x c



x x x x x # x x x x x



Solution:

- Use a new symbol such as 'x'
- Replace each symbol into an x after it has been examined

a b b a c # a b b a c



x b b a c # x b b a c



x x b a c # x x b a c



x x x a c # x x x a c



x x x x c # x x x x c



x x x x x # x x x x x



Problem:

Can we do it non-destructively? i.e. without losing the original strings?

Solution:

Replace each unique symbol with another unique symbol instead of replacing all with the same symbol



Problem:

Can we do it non-destructively? i.e. without losing the original strings?

Solution:

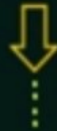
Replace each unique symbol with another unique symbol instead of replacing all with the same symbol

Eg. $a \rightarrow p$

$b \rightarrow q$

$c \rightarrow r$

a b b a c # a b b a c



p q q p r # p q q p r



Problem:

Can we do it non-destructively? i.e. without losing the original strings?

Solution:

Replace each unique symbol with another unique symbol instead of replacing all with the same symbol

Eg. $a \rightarrow p$

$b \rightarrow q$

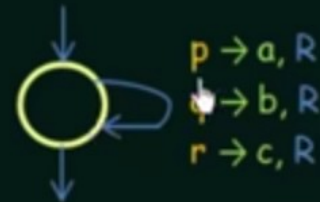
$c \rightarrow r$

a b b a c # a b b a c



p q q p r # p q q p r

Restore the original strings if required



The Universal Turing Machine

The Language

$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a Turing Machine and } M \text{ accepts } w \}$

is Turing Recognizable

Given the description of a TM and some input, can we determine whether the machine accepts it?

- Just Simulate/ Run the TM on the input

M Accepts w: Our Algorithm will Halt & Accept

M Rejects w: Our Algorithm will Halt & Reject.

M Loops on w: Our Algorithm will not Halt.



The Universal Turing Machine

Input: M = the description of some TM

w = an input string for M

Action: - Simulate M

- Behave just like M would (may accept, reject or loop)



The Universal Turing Machine

Input: M = the description of some TM

w = an input string for M

Action: - Simulate M

- Behave just like M would (may accept, reject or loop)

The UTM is a recognizer (but not a decider) for

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$$



Halting problem

⇒ Halting problem is undecidability, It is not a problem, it just asks question: "is it possible to tell whether a given m/c will halt for some given i/p".

Eg input: A T.M & i/p string w

Problem: Does the TM finish computing of the string w in a finite no. of steps? The answer must be Yes or no

Proof: Assume T.M exists to solve this problem & then

⇒ Halting problem is undecidability, It is not a problem
asks question:- "is it possible to tell whether a given m/c
will halt for some given i/p"

$$\Sigma = \{w\}$$

Eg input: A T.M & i/p string w

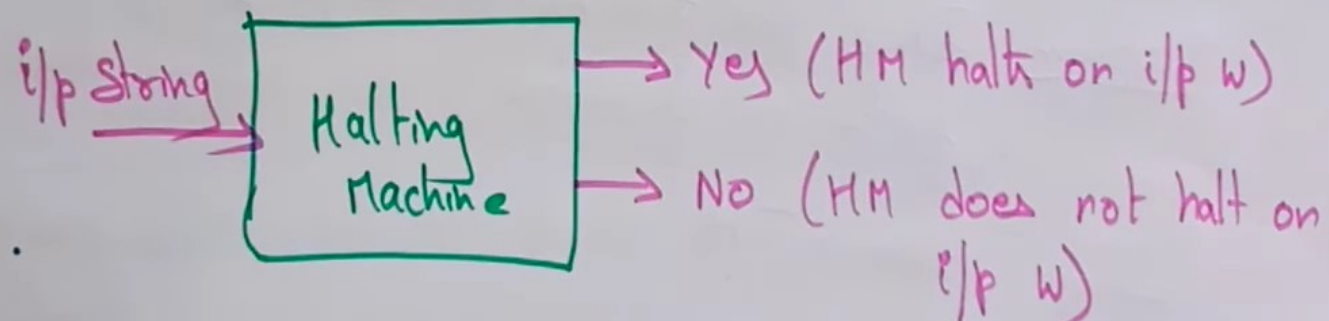
Problem: Does the TM finish computing of the string w in a
finite no. of steps? The answer must be yes or no.

Proof:- Assume T.M exists to solve this problem & then
we will show it is contradicting itself.

We will call this T.M as a Halting m/c that produce a 'yes' or 'no' in a finite amount of time.

→ If the halting m/c finishes in a finite amount of time o/p comes as 'yes' otherwise as 'no'.

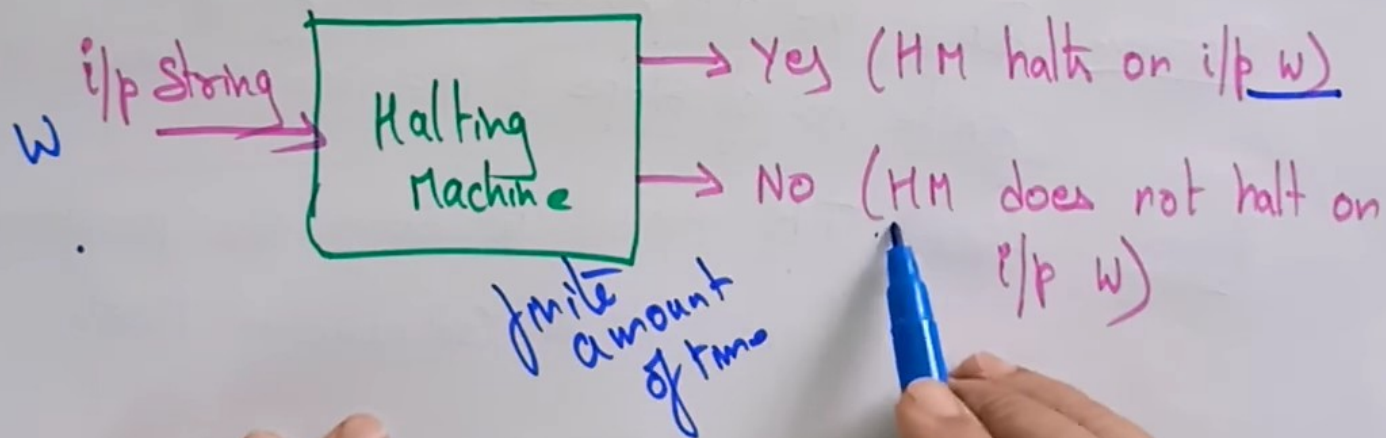
the block dig of Halting m/c :-



'yes' or 'no' in a finite amount of time.

If the halting m/c finishes in a finite amount of time the o/p comes as 'yes' otherwise as 'no'.

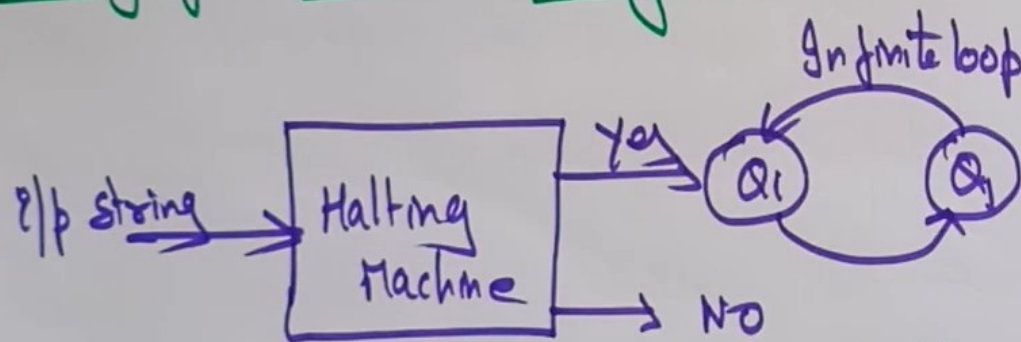
the block dig of Halting m/c :-



Now we will design an inverted halting machine as:-

- If H returns Yes, then loop forever
- If H returns No, then Halt.

The diag for inverted Halting m/c :-

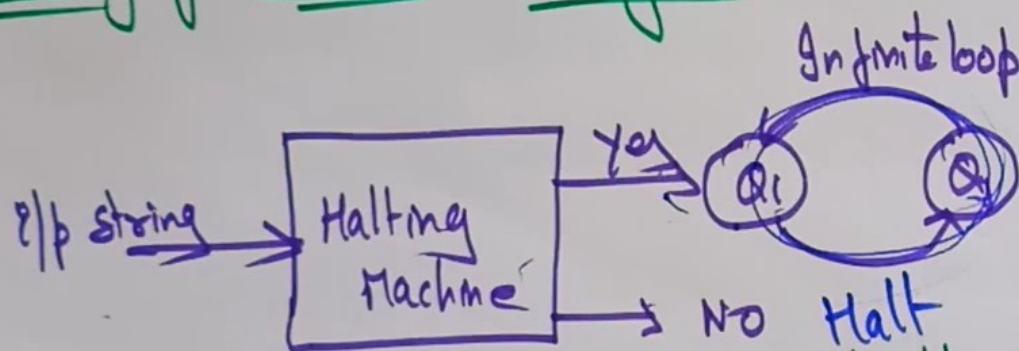


After that a m/c $(HM)_2$ which i/p itself constructed as:-

- If $(HM)_2$ halts on i/p, loop forever

- If H returns Yes, then loop forever
- If H returns No, then Halt.

the diag for inverted Halting m/c :-



After that a m/c $(H)_2$ which i/p itself constructed as :-

- If $(H)_2$ halts on i/p, loop forever
- else, halt.

After this we got a contradiction then the halting prob is undecidable.

Binary strings - end - 0

11010110 = 0 ✓
 ↑
 = 1 ✗

Accepts all valid Java codes



Eg. Compilers

↓
binary
↓
valid? /
invalid ✓

Accepts all valid Java
codes and
never goes into infinite
loop.





Pritam Caxanova 2 months ago

Why can't we design a machine that can't reject infinite loops?

REPLY

11010110 = 0 ✓



DheerajMahraMusic 2 weeks ago

Can someone explain why it's not possible to develop a machine that doesn't allow an infinite loop?

REPLY

Accepts all valid Java codes



chandra singh 2 months ago

why cant we design something which doesnt allow our code to go into infinite loops? I am still not clear

REPLY



Parkash Khatri 6 months ago

Sir

can you please explain that how does it possible to to design a machine that accepts all valid java code and never goes into infinite loop?

I mean if code is valid then it will never go to infinity and vice versa.

please explain.

Thanks.

Show less

REPLY

Accepts all valid Java codes and never goes into infinite loop.



The Halting Problem

Given a Program, WILL IT HALT ?

Given a Turing Machine, will it halt when run on some particular given input string?

Given some program written in some language (Java/C/ etc.) will it ever get into an infinite loop or will it always terminate?

Answer:

- In General we can't always know.
- The best we can do is run the program and see whether it halts.
- For many programs we can see that it will always halt or sometimes loop

BUT FOR PROGRAMS IN GENERAL THE QUESTION IS UNDECIDABLE.



Undecidability of the Halting Problem

Given a Program, WILL IT HALT ?

Can we design a machine which if given a program can find out or decide if that program will always halt or not halt on a particular input?

Let us assume that we can:

$H(P, I)$

├── Halt
└── Not Halt

This allows us to write another Program:

$C(X)$

```
if {  $H(X, X) == \text{Halt}$  }  
    Loop Forever;  
else  
    Return;
```



Let us assume that we can:

$H(P, I)$

├── Halt
└── Not Halt

This allows us to write another Program:

$C(X)$

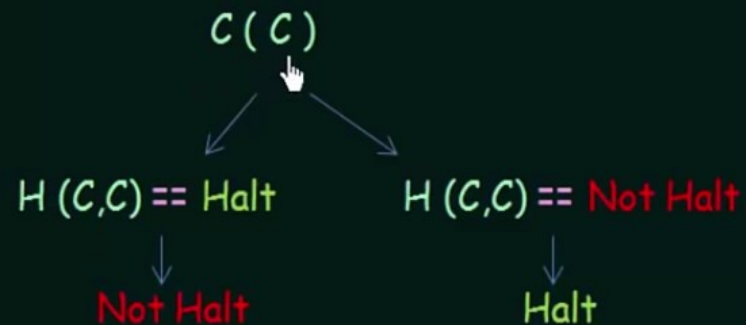
if { $H(X, X) == \text{Halt}$ }

Loop Forever;

else

Return;

If we run 'C' on itself:



Complexity classes

⇒ Time Complexity

- How long computation takes to execute
- In T.M, this could be measured as no: of moves which are required to perform computation.
- no: of m/c cycles

⇒ Space Complexity

- How much storage is required for computation

Complexity classes

Time space

⇒ Time Complexity

- How long computation takes to execute
- In T.M, this could be measured as no: of moves which are required to perform computation.
- no: of m/c cycles

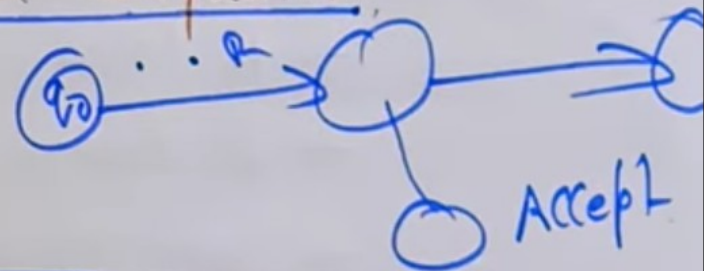
⇒ space complexity.

- How much storage is required for computation

Time Complexity

- How long computation takes to execute how long?
- In T.M this could be measured as no: of moves which are required to perform computation

no: of m/c cycles



Space Complexity

- How much storage is required for computation
- In T.M, no: of cells are used
- no: of bytes used.

Types of complexity classes

① P-class: set of decision problem is solvable in polynomial time or in the class P.

If there exists an algorithm A such that

→ A takes instance of D as i/p

→ A always o/p's the correct answer "yes" or "no"

→ There exists a polynomial p such that the execution of A on i/p of size n always terminates in $p(n)$ or fewer steps

eg: The minimum spanning tree problem is in class P.

The class P is often considered as synonymous with the class of computationally feasible problems, although in practice this is somewhat ~~see~~ unrealistic.

1) NP class:

A decision problem is non deterministically polynomial time solvable or in the class NP if there exists an Algorithm A such that.

→ There exists a polynomial p such that for each potential witnesses of each instance of size n of D , the execution of the algorithm A takes at most $p(n)$ steps

→ Think of a non-deterministic computer as a computer that magically "guesses" a solution, then has to verify that it is correct.

- If sol exists, computer always guesses it
- one way to imagine it: a parallel computer that can freely spawn an infinite no. of processes

→ There exists a polynomial time algorithm A such that for every instance of size n of D , the execution of the algorithm A takes at most $p(n)$ steps

→ Think of a non-deterministic computer as a computer that magically "guesses" a solution, then has to verify that it is correct.

- If sol exists, computer always guesses it
- one way to imagine it: a parallel computer that can freely spawn an infinite no. of processes
- Have one processor work on each possible s

- All processors attempt to verify that their sol works
 - If a processor finds it has a working soln o.
- So: NP = problem verifiable in polynomial time.

→ Every problem in this class can be solvable in exponential time using exhaustive search

P versus NP

- Every decision problem that is solvable by a deterministic poly time algorithm is also solvable by poly time non-deterministic alg.
- All prob in P can be solvable with polynomial time alg, whereas all problem in NP-P are intractable.
- It is not known whether $P=NP$. However, many problems known in NP with the property that if they belong to P, then it can be proved that $P=NP$.

$\Rightarrow$ If $P \neq NP$ there are problems in NP that are neither in P nor in NP-complete.

$\Rightarrow$ The problems belongs to class P if it's easy to find a sol for the problem. The problem belongs to NP, if it's easy to check a solution that may have been very tedious to find.